

LLM Foundations for Harness Engineering

A practitioner's textbook on the mechanisms and behavioral boundaries of large language models.

18 sections · generated from Markdown sources

Preface

Why This Book

Large language models are easy to call through an API, but a chat interface hides the mechanisms that shape their behavior. Tokenization changes what the model receives. Training determines what patterns are stored in its parameters. A finite context limits what can influence a response. Decoding turns next-token probabilities into generated text. Without a working model of these mechanisms, it is easy to mistake fluent output for current knowledge, reliable reasoning, or verified fact.

This book explains those mechanisms and the boundaries they create. Its purpose is not to make every model response predictable. It is to make model behavior less mysterious and to give readers a precise vocabulary for reasoning about it.

Goals and Audience

This book is for software engineers and technical readers who use large language models. It follows an engineering-oriented path through LLM fundamentals: tokens, next-token prediction, Transformer attention, training data and scaling, inference and decoding, post-training, prompting, context, knowledge limits, retrieval, tool calls, and evaluation.

By the end, readers should be able to explain model behavior in terms of tokens, training, context, decoding, and external evidence. They should also be able to distinguish a property of the model from a property of the application around it.

Source Material

The primary teaching material is Andrej Karpathy's [Intro to Large Language Models](#) and [Deep Dive into LLMs like ChatGPT](#). The first lecture frames an LLM as two artifacts: parameters and the code that runs them. The second expands this picture to cover data, tokenization, training, inference, post-training, and practical mental models. This book turns those lectures into a written sequence and adds references to foundational papers where they anchor specific concepts.

Prerequisites and Non-Goals

Readers should be comfortable with basic software engineering ideas such as APIs, tests, and logs. No prior deep-learning mathematics is required. When mathematics is useful, the book introduces only what is needed to understand the mechanism and its consequences.

This book does not teach readers how to train a frontier model, compare every current model provider, or survey every alignment method. Those targets change quickly or require substantially more depth. The focus here is a durable foundation for understanding the models that engineers and technical readers encounter in practice.

Stable Properties of Language Models

Several model-side facts organize the chapters that follow:

- A language model receives and produces tokens rather than words or meanings directly.
- Its context is finite and is not persistent memory.
- Knowledge learned during training is distributed through parameters; those parameters are not a current, authoritative database.
- Decoding rules select from next-token probabilities, so the way generation is configured is part of the resulting behavior.

Later chapters refine these statements, show where their simple formulations break down, and explain how external evidence can enter a model's context without becoming knowledge stored in its parameters.

Companion Book

The companion book, [Agent Harness: A Practitioner's Textbook](#), covers the systems built around a model, including context management, tools, sandboxing, workflow patterns, and evaluation. This book establishes the model

mechanisms and behavioral boundaries that those system designs must account for. Readers can use either book independently and follow the companion links when a topic crosses that boundary.

For the chapter map and suggested reading path for this book, see the [README](#).

Chapter 1: The LLM as a Token Machine

The most useful first approximation is simple: a large language model receives a sequence of tokens and produces probabilities for what token could come next. It does not directly see words, files, websites, or users. It receives token IDs that encode a representation of its current context.

Tokens In, Probabilities Out

For a given token context, the model computes a score for every token in its vocabulary. The runtime converts those scores into a probability distribution over the next token. In this sense, an LLM is a parameterized function from token sequences to next-token probabilities.

Karpathy's short introduction uses a deliberately demystifying frame: a trained model can be thought of as two files, one containing parameters and one containing code that knows how to run them ([Intro to LLMs, around 00:00:24](#)). Real implementations contain more machinery, but the approximation exposes the essential interface: token context goes in, and a distribution over the next token comes out.

Generating a Sequence

One pass does not normally produce a complete answer. Generation repeats the same basic operation:

1. Evaluate the current token context.
2. Produce probabilities for the next token.
3. Select a token according to a decoding rule.
4. Append that token to the context.
5. Repeat until a stop condition is reached.

The word *select* matters. Greedy decoding chooses the highest-probability token, while stochastic decoding samples from a distribution. Both are decoding rules, but greedy selection is not sampling. [Inference and sampling](#) develops this loop and its stopping conditions in detail.

Architecture, Parameters, Runtime, and Application

Four layers are often blurred together:

- **Architecture** defines the shape of the computation, such as a decoder-only Transformer.
- **Parameters** are the learned numerical values that specialize that architecture.
- **Runtime** loads the parameters, evaluates the model, and applies a decoding rule.
- **Application layer** prepares inputs, calls the runtime, and interprets or presents its outputs.

Two models may share an architecture but contain different parameters. Conversely, two applications may call the same checkpoint yet produce different user experiences because they prepare different contexts or interpret the returned tokens differently.

Karpathy makes the parameters concrete by describing them as ordinary files that can be copied, downloaded, and loaded by runtime code ([Intro to LLMs, around 00:01:35](#)). Their storage size is roughly determined by parameter count and numerical precision. A 70-billion-parameter model stored at two bytes per parameter requires about 140 GB for the parameters alone, before runtime overhead.

Where Parameters Come From

Parameters are produced by training, not by the inference loop. During pretraining, optimization repeatedly changes the parameters to reduce next-token prediction error across a large collection of token sequences. Creating a large checkpoint can require substantial data, compute, and time, even though running an existing checkpoint is a more routine operation ([Intro to LLMs, around 00:03:59, 00:05:16](#)).

[Post-training](#) starts from a pretrained checkpoint and further changes learned parameters—or adds learned adapter parameters—to shape tendencies such as instruction following, answer formatting, and refusal. The result is an updated checkpoint or an additional set of learned weights, not the original parameter file left unchanged.

Most model users do not create a base checkpoint themselves. They call an existing checkpoint through an API, an inference provider, or a local runtime. Regardless of where inference runs, the same separation holds: training produces learned parameters; the runtime uses them to compute next-token probabilities.

Parameters Are Not a Database

Parameters encode distributed statistical structure and learned behavior. They are not rows that can be queried by key, inspected as exact records, or updated one fact at a time. A model may reproduce facts and patterns learned during training, but the parameter file is not an authoritative database and does not carry built-in provenance for each generated claim.

This distinction is about representation, not about whether a model can answer factual questions. Fluent recall can emerge from learned parameters without turning those parameters into a record store.

Tokens Do Not Cause External Side Effects

The model's direct output is tokens. A continuation may describe reading a file, sending a message, or taking another action, but generating that description does not make the action happen. External software must interpret the output and perform any operation.

[Reasoning, tools, and agents](#) returns to this boundary: a model can generate a proposed call, while an external system executes it and may return an observation. The proposal and the external effect remain different events.

A Model Call Is Not the Whole Application

Karpathy's deep dive begins by asking what is behind the text box and where its generated words come from ([Deep Dive, around 00:00:28](#)). At the model boundary, the answer is the token-generation loop. At the application boundary, there is also software that constructs the input, invokes the runtime, and handles the output.

It is still useful to discuss *model behavior*: the phrase can refer to learned tendencies, probability distributions, or generated sequences under specified conditions. But observable application behavior may also include rendering, stored state, and external effects. Those additional outcomes are not produced by the model call alone.

The next chapters deepen the model-side picture: [tokenization](#) explains how text becomes token IDs, [next-token prediction](#) explains the training objective, [Transformer attention](#) explains the core architecture, and [inference and sampling](#) explains decoding. System design beyond the model boundary belongs to the companion [Agent Harness](#), especially its chapters on [the harness boundary](#), [state](#), [permissions and side effects](#), and [evaluation](#).

Key Takeaways

- An LLM maps a token context to probabilities for the next token.
- Sequence generation repeats next-token prediction and token selection until a stop condition.
- Architecture, parameters, runtime, and application are distinct layers.
- Pretraining and post-training produce learned parameters; inference uses them.
- Parameters are distributed learned representations, not a queryable database.
- Model-generated tokens do not by themselves cause external actions.

Chapter 2: Tokenization

Before a language model can process text, a tokenizer converts the text into a sequence of integer token IDs. A token may correspond to a whole word, part of a word, punctuation, whitespace together with nearby characters, a byte, or another learned unit. The boundaries are properties of a particular tokenizer, not universal linguistic boundaries.

Karpathy spends time on tokenizer examples because they explain otherwise surprising behavior: leading spaces can change token IDs, capitalization can change segmentation, and rare strings can expand into many pieces ([Deep Dive, around 00:12:07](#)). The direct result of tokenization is sequence length. That length then affects how much text fits in a context window and, depending on the model and service, how much computation, latency, or billed usage a request requires.

Text to Token IDs

At a high level, the input path is:

text → token pieces → token IDs → embedding vectors

The tokenizer performs the first two transformations. It segments text according to a fixed vocabulary and encoding rules, then looks up the integer ID assigned to each piece. The model uses those IDs to select rows from its learned embedding table. It operates on the resulting vectors, not directly on the original characters.

Decoding follows the reverse mapping: token IDs are mapped back to pieces and joined into text. This does not mean every tokenizer round-trips every input perfectly. Optional normalization may change the text before segmentation, and decoding may omit control tokens. Byte-level tokenizers can preserve ordinary input text exactly when their byte mapping and decoding rules are used consistently.

A tokenizer is part of a model's specification. Two models can assign different pieces, IDs, and sequence lengths to the same string. Token IDs therefore have meaning only with the matching tokenizer and vocabulary.

Why Subword Tokens Exist

A whole-word vocabulary is brittle. Names, inflected words, code identifiers, URLs, scientific notation, emoji, and text from many languages would constantly produce unseen words. A character-only vocabulary can represent such inputs, but it usually creates much longer sequences and gives the model less opportunity to reuse frequent multi-character patterns as single units.

Subword tokenization is a compromise. Frequent strings can receive their own tokens, while less frequent strings are composed from smaller pieces. Sennrich, Haddow, and Birch showed how subword units could support open-vocabulary neural machine translation by decomposing rare and unseen words ([Neural Machine Translation of Rare Words with Subword Units](#)).

Byte Pair Encoding (BPE), WordPiece, and Unigram are related subword approaches, but they do not learn or segment text in exactly the same way. BPE training typically begins with a base inventory and repeatedly adds a token for a selected adjacent pair. The base inventory depends on the tokenizer: classical formulations may begin with characters or symbols, whereas **byte-level BPE** begins with a representation of bytes.

That distinction matters. A byte-level BPE tokenizer that retains the full byte alphabet can encode arbitrary Unicode text through its UTF-8 bytes, falling back to smaller byte units when no larger learned piece matches. It is not correct to claim that every BPE tokenizer starts from raw bytes or that every subword tokenizer is automatically free of unknown tokens. Other tokenizers may use an unknown token, character coverage rules, or a separate byte-fallback mechanism.

How a Tokenizer Is Trained and Used

Tokenizer training happens before the tokenizer is used to encode model inputs. A representative text corpus is collected, and choices are made about normalization, whitespace handling, pre-tokenization, the base alphabet, special tokens, and vocabulary size. The training algorithm then learns a vocabulary and, depending on the method, merge rules or token scores. Once model training begins, this tokenizer is normally fixed so that each token ID continues to identify the same embedding row.

For BPE, training counts adjacent units and repeatedly adds selected merges until it reaches a target vocabulary or another stopping condition. Unigram tokenization instead starts with many candidate pieces, learns probabilities for

them, and prunes the vocabulary while preserving likely segmentations. These procedures both produce subword vocabularies, but the resulting boundaries need not agree.

At encoding time, a tokenizer generally applies some version of these steps:

1. Normalize the text if the tokenizer defines a normalization rule.
2. Divide or mark the input according to its whitespace and pre-tokenization rules.
3. Segment it using the learned vocabulary and algorithm.
4. Replace each resulting piece with its vocabulary ID.
5. Add any special tokens required by the model's input format.

Implementations may combine or omit steps, so the tokenizer itself is the authority. Its decoder must also be used to interpret generated token IDs correctly.

Reading Tokenization Examples

Token boundaries must be inspected with a specific tokenizer. The following are possible segmentations, not claims about every model:

Text	One possible segmentation	What it illustrates
world versus world	different pieces for the two forms	A leading space may be encoded with nearby text.
unhappiness	un + happi + ness	Familiar subword fragments can compose a less frequent word.
HTTPResponse2	a mixture of larger fragments and smaller pieces	Case changes, identifiers, and digits can shift boundaries.
你好	one piece, multiple character pieces, or byte fallback	Language coverage and the tokenizer's base units matter.

Karpathy demonstrates the same principle with capitalization, leading spaces, and small punctuation changes ([Deep Dive, around 00:12:33](#)). Newlines and indentation can likewise change the sequence produced for code. Debugging tools sometimes display artificial markers for spaces or bytes; those markers describe the tokenizer's internal representation and are not necessarily literal characters in the input.

Small formatting changes therefore can produce different token sequences. They do not guarantee a large behavioral change, but the model is receiving a different sequence of IDs.

Token Counts Are Not Word Counts

A model's sequence limits are measured in tokens, not words, files, messages, or characters. A rough characters-per-token estimate can be useful for a quick approximation in familiar English prose, but it is not a reliable conversion rule.

Code, tables, numbers, URLs, repeated whitespace, uncommon symbols, and different writing systems can all segment differently. The result also depends on how well a language or text style was represented when the tokenizer was trained. It is therefore unsafe to infer that two passages with the same character or word count will occupy the same number of tokens.

For an exact count, encode the actual text with the tokenizer for the target model and count the returned IDs. A context window usually has to accommodate both the input and generated continuation; [Chapter 9](#) explains that shared sequence limit in detail. Computation and pricing policies are model- and provider-specific consequences of these counts, not properties of tokenization alone.

Text token counts also cannot be used to estimate image, audio, or other multimodal inputs. Those inputs use model-specific encoders and accounting rules.

Special Tokens and Chat Templates

Special tokens are vocabulary entries reserved for structural or control purposes. Depending on the model, they may mark the beginning or end of a sequence, separate documents, or delimit messages and roles. They are still token IDs, but their interpretation comes from the model's training format rather than from ordinary written language.

A chat API may accept a list of messages, but the model ultimately receives an encoded sequence. A **chat template** renders message roles and contents into the format expected by that model, often inserting special tokens and fixed separators. Assistant models learn these conventions during [post-training](#).

Karpathy returns to tokenization later in the deep dive to show that a conversation, like any other model input, must be represented as tokens ([Deep Dive, around 01:05:03](#)). The rendered template creates token overhead that is not visible in message content. Counting only the `content` fields—even with the correct text tokenizer—can therefore undercount a request. Exact accounting must use the target model's chat template together with its tokenizer; the format and overhead can change across model families or versions.

Character-Level Capability Gaps

Tokenization does not present most text as one model position per character. A word may occupy one token, several subword tokens, or a sequence that falls back to characters or bytes. As a result, operations that require stable access to individual characters—counting letters, reversing an unfamiliar string, or tracking exact spelling—can be harder than their surface simplicity suggests.

This is a tendency, not an absolute inability. Models can learn spelling patterns, infer characters contained inside tokens, and sometimes solve these tasks correctly. Tokenization is nevertheless one reason the representation is poorly aligned with character-by-character operations. Karpathy uses spelling and letter-counting examples to illustrate this mismatch ([Deep Dive, around 02:01:11](#)).

Numbers can also be split into variable-length pieces, but token boundaries are not a complete explanation for arithmetic errors. Training data, the next-token objective, limited learned algorithms, and the difficulty of carrying intermediate state can also contribute.

Common Token-Counting Pitfalls

Several recurring mistakes follow directly from model-specific encoding:

- Estimating from words or characters and treating the estimate as an exact limit.
- Counting with a tokenizer from a different model family or tokenizer version.
- Counting message contents without applying the chat template and its special tokens.
- Measuring only the prompt even though generated tokens occupy part of the available sequence.
- Assuming prose, source code, URLs, and multilingual text have the same token density.
- Applying a text-token estimate to image, audio, or other multimodal input.

Truncation creates a related problem. A character slice may not correspond to the desired token budget, while a token slice can cut through a sentence or other meaningful unit after decoding. Token counts determine whether a sequence fits; they do not determine which content is semantically safe to remove.

Key Takeaways

- A tokenizer maps text into model-specific token IDs; embedding lookup turns those IDs into the vectors processed by the model.
- Subword methods balance reusable frequent pieces against the ability to compose unfamiliar strings.
- BPE is a family of methods, and only byte-level variants necessarily begin from byte representations.
- Sequence length depends on the exact tokenizer, text, special tokens, and chat template—not on word count alone.
- Tokenization can make character-level operations less natural, but it is not the sole cause of every spelling or arithmetic error.

Chapter 3: Next-Token Prediction

The pretraining objective for an autoregressive language model is deceptively simple: given previous tokens, predict the next token. At each position, the model assigns a probability to every token in its vocabulary, and training rewards it for assigning a higher probability to the token that actually follows. Karpathy illustrates this process by taking windows of tokens from a large dataset and repeatedly predicting what comes next ([Deep Dive, around 00:15:36](#)).

This objective supplies the learning signal for pretraining. It does not, by itself, give the model an authoritative store of facts, an intention, or an assistant persona.

The Next-Token Objective

Suppose a training sequence begins:

The capital of France is Paris.

It provides a series of prediction targets:

The	-> capital
The capital	-> of
The capital of	-> France
The capital of France	-> is
The capital of France is	-> Paris

The actual units are tokens rather than words, so the boundaries may differ from this simplified display. The same objective applies to prose, code, dialogue, mathematical derivations, tables, and Markdown. Whatever the text form, the training target is expressed as a continuation token rather than as a separately programmed symbolic rule.

Training Text Becomes Token Sequences

Before training, source text is filtered, transformed, and encoded by a tokenizer. The resulting token IDs are arranged into sequences of a length the model can process. Each sequence supplies many prefix-and-target relationships for training.

In Karpathy's small demonstration, the cleaned dataset is represented as one long array of tokens from which training windows are sampled ([Deep Dive, around 00:14:34](#)). That is a useful teaching device, not a universal description of production training. A production pipeline may preserve document boundaries, insert boundary tokens, split long documents, or pack pieces from multiple documents into fixed-length sequences. The exact construction determines which transitions the model encounters.

The important invariant is that training operates on token sequences with next-token targets. It does not require the entire corpus to become one semantically borderless stream.

Loss and Gradient Updates

For a target token, the loss is its negative log-probability under the model. Assigning a high probability to the correct token produces a low loss; assigning a low probability produces a high loss. Averaging this quantity across predicted positions gives the cross-entropy loss, also called negative log-likelihood in this setting.

Perplexity is the exponential of the average cross-entropy when natural logarithms are used. It puts the loss on an effective-choice scale: a perplexity of 10 corresponds to the same average uncertainty as a uniform choice among 10 possibilities. Karpathy describes loss as the single number the training process tries to reduce ([Deep Dive, around 00:35:58](#)).

The training loop is:

1. Build a batch of token sequences and their next-token targets.
2. Run the model to produce a probability distribution at each predicted position.
3. Compute the average loss for the target tokens.
4. Backpropagate gradients through the model.
5. Use an optimizer to update the parameters.
6. Repeat across many batches.

No one writes a separate rule for every grammar pattern, fact, or coding convention. Gradient updates change many parameters so that continuations resembling the training distribution become more probable.

Memorization and Generalization

The objective does not force a clean choice between memorization and generalization. A model can memorize some exact or near-exact sequences, especially when they are repeated or distinctive. It can also learn regularities shared across many examples and apply them to sequences it did not see during training. That second behavior is generalization.

Karpathy uses compression as an intuition: a model that predicts text well has captured structure in the data ([Deep Dive, around 00:50:22](#)). The analogy is lossy rather than zip-like. Parameters can encode broad patterns, associations, and some exact strings without preserving every source as a recoverable record.

This is also why parameterized knowledge is not an authoritative or automatically current source. It reflects the training distribution and its time coverage, can combine conflicting patterns, and does not inherently retain provenance for a generated claim. A plausible continuation is not the same thing as a verified fact.

Why Prediction Produces Broad Capability

Improving next-token prediction across web pages, books, code, papers, and conversations rewards representations of many latent regularities. Grammar helps predict sentences. Factual associations help predict encyclopedia-like passages. Program structure helps predict code. Dialogue conventions help predict conversations. Arithmetic and reasoning patterns can help predict worked examples.

The GPT-3 paper showed that a sufficiently large autoregressive model can adapt to many tasks from instructions or examples in its context, without a task-specific parameter update ([Language Models are Few-Shot Learners](#)). This is in-context learning, discussed further in [Chapter 8](#).

These results do not establish human-like understanding in every sense, nor do they imply that every capability appears equally or reliably. They show that one predictive objective can produce internal representations that support many downstream behaviors.

Capability Without Intention

Pretraining changes parameters to reduce prediction loss. It does not give the model an intrinsic commitment to being helpful, truthful, or safe. A base model learns statistical patterns of helpful answers, harmful passages, correct explanations, mistakes, and many other kinds of text. Which pattern it continues depends on its learned distribution and the prefix it receives, not on an independently formed intention.

Changing a prefix can therefore change the distribution of likely continuations. That fact is the basis of prompting and in-context learning, but conditioning on a prefix does not install a new training objective or a durable intent.

Base Models and Assistant Models

The direct result of next-token pretraining is a base model. It can complete text, imitate formats, answer some questions, and reproduce many learned patterns. It has not necessarily been optimized to respond consistently as a helpful conversational assistant.

[Post-training](#) further changes the model's output distribution, increasing the probability of behaviors such as instruction following, conversational helpfulness, and learned refusals. Those assistant behaviors are learned response tendencies, not proof of an inner intention and not guarantees of truth or safety.

Key Takeaways

- Autoregressive language models are pretrained to predict each next token from preceding tokens.
- Training uses bounded token sequences; representing the entire dataset as one long stream is an illustrative implementation, not a universal requirement.
- Cross-entropy loss and gradient updates produce both memorization and generalization.
- Broad predictive capability does not create intrinsic intent or authoritative knowledge.
- Base models come from pretraining; assistant behavior is shaped further by post-training.

Chapter 4: Transformer and Attention

Most modern LLMs are based on the Transformer architecture introduced in *Attention Is All You Need* (Vaswani et al., 2017). Karpathy's introduction likewise identifies the Transformer as the neural network architecture behind these models (Intro to LLMs, around 00:11:40). The architecture explains how a sequence of tokens becomes context-dependent predictions.

Inputs and Outputs

A language model begins with token IDs. An embedding table maps each ID to a vector, so an input sequence becomes a sequence of vectors. The model must also represent order: without position information, the same tokens in different orders would be difficult to distinguish.

Transformer families encode position in different ways. Examples include learned absolute position embeddings, sinusoidal encodings, relative position biases, and rotary position embeddings applied within attention. Position is therefore part of the model's representation of the sequence, even though it is not always literally added to the token embedding.

The vectors pass through a stack of Transformer blocks. Each block produces a new representation for every position. After the final block, a learned projection maps the representation at each position to vocabulary logits. During next-token training, those logits are compared with the following token at every eligible position. During generation, the logits at the last position define the distribution used to select the next token.

Karpathy describes this flow as tokens passing through repeated Transformer blocks until the network produces next-token predictions (Deep Dive, around 00:23:24). The model is not parsing the sequence into a set of explicit variables. It is repeatedly transforming vector representations.

Original Transformer and Decoder-Only LLMs

The original Transformer was an encoder-decoder architecture designed for sequence-to-sequence tasks such as machine translation. Its encoder builds representations of an input sequence. Its decoder uses both causal self-attention over the output prefix and cross-attention to the encoder's representations.

Many autoregressive LLMs instead use a decoder-only architecture. They have no separate encoder or cross-attention stage. A single stack processes a prefix under a causal constraint and predicts the token that follows it. In training, many next-token predictions can be computed in parallel because the correct prefix for every position is already present. At generation time, tokens are appended one at a time.

"Transformer" therefore names a family of architectures, not one fixed layout. Model families also vary in normalization placement, position method, attention variants, activation functions, and other details. The shared idea is a stack of learned blocks that turns a token sequence into contextual representations.

Self-Attention

Self-attention lets a position mix information from other positions in the same sequence. This makes the representation of a token depend on its context. For example, a word in a question can interact with words in the source text that precedes it, and a line of code can interact with an earlier variable definition.

The attention computation is learned and soft. It does not retrieve a symbolic record or copy a single fact through a fixed rule. Instead, it creates weighted mixtures of value vectors, which are then combined with other signals and transformed by the rest of the network.

Queries, Keys, and Values

Within one attention head, learned projections turn each input representation into three vectors:

- A **query** represents what the current position is looking for.
- A **key** represents how an available position can be matched.
- A **value** represents information that the available position can contribute.

For an input matrix (X), one head computes projections ($Q=XW_Q$), ($K=XW_K$), and ($V=XW_V$). Its output has the form

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right) V$$

where (M) is the attention mask. The dot products produce compatibility scores, the softmax turns the permitted scores into weights, and the weighted sum mixes the value vectors.

Suppose a code sequence contains `user_count = ...` and later reaches `return`. Some heads may give the earlier definition relatively high weight and contribute information about `user_count` to the later representation. That is only an intuition for one intermediate computation: no single head normally decides that the next token will be `user_count`. The outputs of multiple heads, residual paths, MLPs, and many layers all contribute before vocabulary logits are produced.

The same caution applies to prose. A position used to answer a question may weight an earlier sentence that states a relevant fact, but the final answer does not follow mechanically from one attention link. Attention can mix useful, irrelevant, or misleading information, and high attention weight alone is not proof of causal importance.

Causal Masking

In a decoder-only language model, the mask (M) prevents a position from attending to later positions. Disallowed scores are effectively set to negative infinity before the softmax, so their weights become zero. A representation at position (i) can use tokens at positions up to (i), but not tokens that come after it.

This causal mask aligns the architecture with next-token prediction. During training, the model can process a complete sequence while still preventing each position from seeing the token it is supposed to predict. During generation, the same constraint means that a newly generated token depends only on the existing prefix.

Multi-Head Attention

Multi-head attention runs several attention computations with different learned projections. Their outputs are concatenated and projected back into the model's residual stream. Multiple heads give a layer several ways to combine information across positions in parallel.

Heads sometimes exhibit recognizable patterns, such as attending locally or tracking repeated tokens, but they are not guaranteed to divide into clean human-readable roles. Their contributions can overlap, change with context, and interact with later layers. A head is best understood as one component of a distributed computation rather than an independent decision-maker.

The attention block is the mechanism Karpathy highlights when explaining how positions communicate inside a Transformer ([Deep Dive, around 00:24:29](#)). It gives earlier text a path to affect later representations, but it does not guarantee that the model will identify or use the most relevant information correctly.

The Transformer Block

Attention is only one part of a Transformer block. A typical decoder-only block contains:

1. A normalization operation.
2. Causal multi-head self-attention.
3. A residual connection that adds the attention output back to the running representation.
4. Another normalization operation.
5. A feed-forward network, often called an MLP, applied independently at each position.
6. Another residual connection.

Exact ordering differs across model families; for example, normalization may appear before or after a sublayer. The functional distinction is more important than the exact variant: attention mixes information across token positions, while the MLP applies learned nonlinear transformations within each position. Residual connections preserve and accumulate signals as representations pass through many blocks, and normalization helps keep those computations stable.

Repeating these blocks produces representations conditioned on the available prefix. Individual layers need not correspond to clean stages such as "syntax first, meaning later." Features can be distributed across layers and components, and the same component may participate in different computations for different inputs.

Distributed Representation and Interpretability

A model's learned knowledge and behavior are distributed across many parameters and activations. Karpathy emphasizes that billions of parameters collaborate in ways that are not yet fully understood mechanistically ([Intro to LLMs, around 00:11:57](#)). Asking where one fact is stored therefore rarely has a simple answer.

The same is true of a particular prediction. Attention weights reveal part of an internal computation, but they are not a complete explanation: value vectors, MLPs, residual paths, and later layers also matter. Mechanistic interpretability studies these computations, yet current methods do not generally provide a simple, exhaustive causal account of every generated token.

Distributed representation also means that model knowledge is not an editable table of facts. Changing a parameter can affect many inputs, while one behavior may depend on many parameters. The network still performs a definite numerical computation, but its internal concepts usually do not map one-to-one to human labels.

Architecture and Training Are Separate

Architecture defines the operations the network can perform; training determines the parameter values used by those operations. A randomly initialized Transformer has the architecture but not useful language behavior. Pretraining fits the parameters to next-token prediction over training sequences, and post-training can further change the distribution of its responses.

This distinction prevents architecture from becoming a catch-all explanation. Self-attention provides a path for earlier text to influence a later prediction, but training determines which patterns the model has learned to recognize and how reliably it uses them. Two models with similar architectures can behave differently because their data, objectives, parameter counts, and post-training differ.

Attention Cost

For standard full-sequence attention on (n) tokens, each position forms scores against the other permitted positions. The score matrix therefore has $(O(n^2))$ entries per head. Holding model width and layer count fixed, the sequence-length-dependent attention work is quadratic during training and during prompt processing, also called **prefill**. A causal mask removes future-position interactions but does not change this asymptotic scaling.

This statement applies specifically to the attention part of a block. Linear projections and MLPs also consume substantial compute, and for many practical sequence lengths they may dominate total runtime. Conventional attention implementations may also require $(O(n^2))$ temporary memory for attention scores, while memory-efficient kernels can avoid materializing the full matrix without eliminating the underlying pairwise attention work.

Autoregressive decoding has a different profile when a **KV cache** is used. The keys and values for existing tokens are stored at every layer. For one new token, the model computes only the new query, key, and value; that query attends over the cached prefix. Holding model dimensions fixed, the attention work for that decode step grows as $(O(n))$ with prefix length rather than recomputing $(O(n^2))$ attention for the entire prefix. KV-cache memory grows linearly with the number of cached tokens.

Across many generated tokens, those linear-per-step costs accumulate: generating (m) tokens after a prompt of length (p) requires attention score work proportional to roughly $(mp + m^2)$, in addition to prefill and the other computations in each block. Architecture and kernel optimizations can change constants or use approximate/sparse patterns, but longer sequences still impose real compute, memory, and latency costs.

Key Takeaways

- A decoder-only Transformer maps token and position information through repeated blocks to next-token logits.
- Causal self-attention mixes information from the available prefix through learned query, key, and value projections.
- Multiple heads and layers contribute distributed intermediate signals; a single attention head does not directly choose the output token.
- Attention can make relevant earlier text influential, but it does not guarantee correct selection or use of that information.
- Standard prefill attention is quadratic in sequence length, while KV-cached decoding uses linear attention work per new token and linear cache memory.

Chapter 5: Training Data and Scaling

Pretraining quality depends on the distribution of training data, the number and architecture of model parameters, and the compute used to optimize them. These factors interact: more parameters are useful only when the training run supplies enough suitable data and compute, while more tokens are useful only to the extent that they add learnable signal. Karpathy's deep dive begins with data collection and filtering because the training distribution is one of the main determinants of a model's capabilities and blind spots ([Deep Dive, around 00:01:28](#)).

Training Data Is a Constructed Distribution

A large pretraining corpus may include web pages, books, code, papers, discussions, and documentation. It may also contain duplicated templates, spam, broken text extraction, outdated claims, toxic material, personal data, and strong biases toward some languages and domains. A corpus is therefore not simply "the web." It is the output of a pipeline that turns collected sources into a distribution of token sequences.

FineWeb is a public example of this process. The deep dive uses it to show how Common Crawl-derived pages are transformed into a web-scale text dataset, with about 15 trillion tokens in the version discussed ([Deep Dive, around 00:01:28](#)). Raw pages contain navigation menus, cookie banners, boilerplate, repeated mirrors, and extraction errors as well as useful text.

Dataset construction commonly includes:

- collecting snapshots and other source corpora;
- extracting and normalizing text;
- identifying language and content type;
- filtering low-quality, unwanted, or sensitive material;
- removing exact and near duplicates;
- assigning sampling weights to different sources;
- and tokenizing and packing the selected text for training.

The resulting distribution is deliberately constructed. It is neither a uniform sample of human knowledge nor a direct representation of truth.

Filtering, Mixing, and Policy

Filters can be hard rules, statistical classifiers, or model-based quality scores. Every threshold creates tradeoffs. A strict quality filter may remove spam but also discard unusual dialects or specialist material; a loose filter may preserve breadth while admitting more noise. Karpathy uses language filtering as a concrete example: an English-focused dataset reduces non-English material by design ([Deep Dive, around 00:04:54](#)).

After filtering, sources are mixed in chosen proportions. The mixture need not match the amount of raw data available. Code, mathematics, books, or a low-resource language can be upweighted; a very large but noisy web source can be downweighted. Upweighting a limited source may also cause its examples to be repeated more often during training.

These choices embed policy in the data. They influence which languages, domains, styles, viewpoints, and behavior patterns contribute most often to the gradient. As a result, competence is uneven: strong performance in one domain does not imply equally strong performance in another.

Deduplication, Frequency, and Memorization

Repeated sequences receive repeated training weight. Exact copies, near-duplicate pages, quoted passages, and syndicated content can therefore make some text much more frequent than its apparent semantic importance would suggest.

Deduplication can operate at document, passage, or shorter-span level. It improves data efficiency and can reduce both verbatim memorization and overlap between training and evaluation data. It is not perfect: paraphrases and partially copied documents are difficult to identify, and some repeated structures are legitimate.

Memorization is correspondingly uneven rather than all-or-nothing. A model may reproduce a distinctive sequence that occurred many times while failing to recall a fact that appeared only rarely. Frequency is not the only factor—model size, context, optimization, and properties of the sequence also matter—and models can generalize patterns without storing exact copies. Deduplication changes the odds of memorization; it does not eliminate it.

Coverage, Freshness, and Competence

Coverage has several dimensions: topic, language, time period, genre, code ecosystem, cultural setting, and level of difficulty. A model can sometimes generalize beyond examples seen in training, but sparse or poor-quality coverage usually provides less evidence from which to learn. Non-public information may have no representation in the corpus at all.

Collection dates are bounded and uneven. A nominal training cutoff is useful shorthand, but it should not be imagined as one clean timestamp: different sources may have been collected at different times, and pretraining and post-training datasets may have different date ranges. Once a checkpoint is trained, its parameters do not automatically change as the world changes.

Data freshness is therefore distinct from reasoning ability. Additional reasoning cannot reveal an event absent from the available information. Post-training can further shape which knowledge is expressed and how the model responds, but it cannot make coverage uniform.

Scaling Laws

Across broad experimental ranges, language-model loss has followed approximate power-law relationships with parameter count, dataset size, and training compute. On log-log plots, the reducible part of loss often declines roughly along a straight line as one of these resources increases ([Scaling Laws for Neural Language Models](#)). This implies predictable but diminishing gains: multiplying a resource by a fixed factor tends to multiply excess loss by a roughly fixed factor, rather than producing unlimited improvement.

Scaling variables are coupled. A larger model can be undertrained if it sees too few tokens or receives too little optimization compute. A very large dataset cannot be fully exploited by a model or training run without sufficient capacity and compute. Data quality and mixture also affect the loss reached at a given scale.

Compute-Optimal Training

For a dense Transformer, a useful first approximation is that training compute grows with the product of parameter count and the number of training tokens. Under a fixed compute budget, model size and token count must therefore be balanced.

The Chinchilla study found that many earlier large models were undertrained relative to their parameter count. Within its experimental setting, training a smaller model on more tokens could achieve lower loss at the same compute budget ([Training Compute-Optimal Large Language Models](#)). Its often-cited estimate of roughly 20 training tokens per parameter is a result tied to particular models, data, and assumptions—not a universal constant. Architecture, data quality, repeated data, optimization, and the intended objective can all shift the best allocation.

"Compute-optimal" must therefore name the objective being optimized. The Chinchilla-style question asks how to minimize pretraining loss for a fixed training-compute budget. It does not by itself minimize latency, memory use, or the total compute consumed after deployment.

Training Cost and Inference Cost

Training spends a large amount of compute to produce a checkpoint. Inference spends additional compute each time that checkpoint processes input and generates output. A simplified lifetime accounting is:

$$C_{\text{total}} = C_{\text{train}} + Q * C_{\text{inference_per_request}}$$

where (Q) is the number of requests. The actual terms depend on parameter count, architecture, input and output lengths, hardware, numerical precision, and batching.

This distinction can change the preferred parameter-and-token allocation. When a checkpoint will be used many times, extra training of a smaller model can sometimes trade higher one-time cost for lower repeated inference cost. With few requests, or when the larger model's quality is required, the tradeoff can be different. Training-optimal and lifetime-compute-optimal are separate objectives.

What Scaling Claims Do and Do Not Say

Scaling laws describe aggregate trends such as held-out cross-entropy loss. They do not guarantee that every task, benchmark, language, or capability improves smoothly. A thresholded metric can turn a gradual change in underlying

performance into an apparently sudden "emergent" jump ([Are Emergent Abilities of Large Language Models a Mirage?](#)).

Observed behavior also depends on data mixture, architecture, tokenizer, optimization, and post-training—not parameter count alone. A larger model does not acquire information collected after its training data, and lower average loss does not imply uniformly better behavior. Empirical scaling relationships are strongest within the regimes in which they were measured; extrapolating far beyond them requires caution.

Contamination

Benchmark contamination occurs when evaluation examples, close variants, answer keys, or detailed solution discussions appear in training data. Overlap can enter through pretraining or post-training, especially when public benchmarks and their solutions are widely copied online.

Contamination can make a score reflect recall of evaluation material as well as generalization to unseen cases. Ordinary deduplication does not fully solve the problem: removing repeated training documents is different from comparing training data against a benchmark and its variants. Contamination does not automatically invalidate every result, but it limits what a benchmark score alone can establish about generalization.

Key Takeaways

- Training data is a constructed distribution shaped by collection, filtering, mixing, and policy choices.
- Deduplication changes effective frequency and memorization risk but cannot remove all overlap or memorization.
- Coverage varies across domains, languages, and time, and checkpoint parameters do not update automatically.
- Scaling laws describe useful aggregate trends, while compute-optimal training balances model size and training tokens for a stated objective.
- Training cost, repeated inference cost, and benchmark contamination set important boundaries on scaling claims.

Chapter 6: Inference and Sampling

Inference is the use of a trained model to compute outputs without updating its parameters. Given a token context, the model produces a score, called a **logit**, for every token in its vocabulary. A decoding rule turns those scores into a token choice. The chosen token is appended to the context, and the process repeats.

This chapter follows that path from logits to a complete generated sequence.

What Inference Computes

For a context ($x_{1:t}$), a decoder-only Transformer produces a vector of next-token logits (z), with one value for each vocabulary token. Logits are relative scores, not probabilities. Softmax converts them into a probability distribution:

$$p_i = \exp(z_i) / \sum_j \exp(z_j)$$

Higher logits produce higher probabilities, but every probability also depends on the other logits in the vector. Ordinary inference performs forward computation through the existing parameters; it does not run backpropagation or update the checkpoint.

The context can change a model's output without changing its parameters. A fact mentioned earlier in the current input can influence the next-token distribution because the model conditions on that input. This temporary conditioning is different from learning the fact into the weights or storing it as durable memory.

From Logits to a Token

Greedy decoding selects the token with the highest probability. Given exactly the same logits, transformations, and tie-breaking rule, greedy decoding makes the same next-token choice. It can nevertheless produce repetitive or locally attractive continuations because each step commits to one token without considering every possible complete sequence.

Sampling draws a token from a probability distribution instead of always taking the maximum. Temperature commonly modifies the distribution as

$$p_i(T) = \text{softmax}(z_i / T)$$

for ($T > 0$):

- At `temperature=1`, the logits are not rescaled, so softmax yields the model's unmodified next-token distribution.
- For `0 < temperature < 1`, the distribution becomes sharper and high-logit tokens receive more probability.
- For `temperature > 1`, the distribution becomes flatter and lower-logit tokens receive more probability.
- `temperature=0` is not defined by the formula because it would divide by zero. APIs usually interpret it as greedy or argmax decoding, but the exact contract is provider-specific.

Top-p sampling, also called nucleus sampling, first sorts tokens by probability and keeps the smallest set whose cumulative probability reaches at least (p). It then renormalizes and samples from that set. A top-p value of `1` normally removes no tokens; it does not itself make decoding greedy. Nucleus sampling was introduced to avoid sampling from an unreliable long tail while retaining context-dependent diversity ([The Curious Case of Neural Text Degeneration](#)).

Temperature and top-p are often combined, although the precise order and edge-case behavior depend on the implementation. Lower randomness can suit tasks with a narrow expected form, while broader sampling can expose alternative continuations. Neither setting determines whether the content is correct.

The Autoregressive Loop

Generation is autoregressive: every selected token becomes part of the context for the following token. In simplified form:

1. Encode the input context as tokens.
2. Run the model to obtain logits for the next position.
3. Apply any configured logit transformations or constraints.
4. Select a token by greedy decoding or sampling.
5. Append the selected token to the context.

6. Repeat until a stopping condition is reached.

The initial pass over the input is commonly called **prefill**. The subsequent one-token-at-a-time phase is **decode**. Implementations normally reuse a KV cache during decode rather than recomputing all earlier attention keys and values; [Chapter 9](#) explains that distinction.

An early token choice changes the context and therefore changes all later distributions. This is why two sampled completions can diverge substantially after a small initial difference.

Longer outputs require more computation because each additional output token requires another decode step. The cause is not merely that the final text contains more tokens: generation cannot produce token ($t+1$) until token (t) has been selected and appended.

Stopping Generation

A generation can stop for several reasons:

- The model emits an end-of-sequence token.
- The configured maximum number of output tokens is reached.
- A configured stop sequence is encountered.
- The serving API ends generation for another documented, provider-specific reason.

Many APIs return a **finish reason** that distinguishes a normal stop from a token-limit stop or another termination mode. The field names and possible values vary by provider, so they must be interpreted according to that API's contract.

Reaching the maximum output length means the sequence may be truncated. The last characters can look fluent even when a sentence, code block, or structured value is incomplete. A stop sequence is also an external boundary: depending on the API, the matched sequence may be omitted from the returned text. Neither case says anything by itself about the correctness of the preceding content.

Logit Transformations and Constrained Decoding

Decoding can alter or restrict the next-token distribution before selection. Examples include logit bias, repetition or frequency penalties, and masks that assign invalid tokens effectively zero probability.

Constrained decoding applies such restrictions at every step so that the growing token sequence remains a valid prefix under a grammar, JSON schema, regular language, or fixed label set. Because textual units and tokens do not always align, the decoder must determine which token continuations preserve the constraint rather than merely checking one character at a time.

These methods can guarantee that an output belongs to the supported syntactic language, provided the implementation is correct. They cannot guarantee that a JSON value is factually true, that code has the intended behavior, or that one of several forced labels is an adequate answer. A constraint can also hide uncertainty by requiring a choice when the model would otherwise express that none of the options fits.

Constrained generation concerns which token sequences may be emitted. Interpreting a generated action description or executing an external action is a separate boundary covered in [Chapter 12](#).

Randomness and Repeatability

Sampling uses a pseudorandom state, so repeated runs can choose different tokens even when their probability distributions match. If an API exposes a seed, fixing it may improve repeatability, but guarantees are provider-specific and can depend on the model version, backend, and other decoding settings.

Greedy decoding removes sampling randomness, but `temperature=0` does not guarantee byte-for-byte identical responses across all hosted requests. Nominally identical requests can produce slightly different logits because batching, floating-point kernels, hardware, or serving software can change. If two leading tokens are nearly tied, a small numerical difference can change the argmax; that first difference then changes the rest of the autoregressive path.

The important distinction is:

- With exactly the same logits and deterministic tie-breaking, argmax chooses the same token.

- Requests that look identical at the API boundary are not guaranteed to produce exactly the same logits on every serving stack.
- With sampling, repeatability additionally depends on the pseudorandom state and the sampling implementation.

Finally, repeatability is not correctness. A greedy decoder can reproduce the same false answer consistently, while a sampled decoder can produce a correct answer on one run and an incorrect one on another.

Multiple Complete Samples

Instead of generating one completion, an inference procedure can generate (k) complete samples from the same input. Each sample follows its own autoregressive path, so the set may reveal different phrasings, approaches, or candidate solutions.

Multiple samples are useful when the distribution contains several plausible continuations, but they do not identify the best candidate by themselves. Producing (k) candidates also consumes roughly (k) times as many generated tokens unless batching or early stopping changes the amount of work. How candidates are scored, how pass@k is reported, and whether an evaluator is trustworthy are evaluation questions covered in [Chapter 13](#).

Key Takeaways

- Inference maps a token context to next-token logits without updating model parameters.
- Greedy decoding selects an argmax; sampling draws from a distribution shaped by controls such as temperature and top-p.
- `temperature=1` leaves logits unscaled, while `temperature=0` is an API convention that usually requests greedy decoding.
- Autoregressive generation appends one selected token at a time and stops on an end token, a limit, a stop sequence, or another documented condition.
- Constrained decoding can enforce syntax, not semantic correctness.
- Lower randomness can improve repeatability, but deterministic output is not necessarily correct and hosted execution may still vary.
- Multiple complete samples explore more of the model's distribution at additional inference cost.

Chapter 7: Post-Training

Pretraining teaches a model to predict tokens across a broad text distribution. Post-training applies additional optimization on narrower data or feedback so that the model more often behaves like an assistant. Karpathy's introduction separates pretraining from fine-tuning and describes the later stages that make a model more useful in dialogue and instruction-following settings ([Intro to LLMs, around 00:14:29](#)).

This distinction helps explain why a base model and an assistant model descended from a similar architecture can behave so differently. Post-training changes the probabilities of possible outputs. It can make helpful responses, refusals, particular styles, and multi-turn conventions more likely, but those tendencies are not formal guarantees of correctness, safety, or completeness.

Supervised Fine-Tuning

Supervised fine-tuning, or SFT, continues training on examples that pair an input with a desired response. The examples may include instructions and answers, conversations, formatting patterns, or domain-specific tasks. The model still learns through token prediction, commonly with the loss focused on the target response tokens, but the new data shifts its output distribution toward the demonstrated behavior.

SFT data can be written by people, generated by other models, or assembled from a mixture of human and synthetic sources. It may also be filtered, revised, or annotated before training. The composition varies by project, so neither exclusively human-written data nor exclusively synthetic data is part of the definition of SFT.

A minimal conversational example looks like this:

User: Explain gradient descent.

Assistant: ...

Training on many such examples makes an assistant-like continuation more likely after a similar prompt. A useful mental model is that the model reproduces patterns present in the selected answers: their tone, level of detail, ways of asking for clarification, and kinds of requests they decline. It does not acquire the intentions or understanding of the people who wrote or selected those answers.

SFT can teach useful task behavior, but successful imitation on the training distribution does not by itself establish that an answer is true or that the behavior will generalize to every new input.

Learned Interaction Conventions

Conversation roles and boundaries must ultimately be represented as tokens. A chat template may serialize role labels, message separators, turn endings, and other control tokens into a sequence. When those patterns occur during post-training, they become part of the distribution the model learns.

As a result, an assistant model may learn conventions such as responding in the assistant role, addressing the current request, asking a clarifying question when information is missing, using familiar formatting, and ending a response at an expected boundary. If the training data contains refusals for certain categories of requests, the model can also learn refusal patterns.

These conventions are statistical regularities rather than a symbolic protocol implemented outside the model. Their reliability depends on the training distribution and the context presented at inference time. A familiar serialization can elicit learned behavior more consistently, while a different or ambiguous serialization may not.

Preference Optimization: RLHF, DPO, and AI Feedback

SFT demonstrates desired responses, but many qualities are easier to express by comparing candidates than by writing a single ideal answer. In the InstructGPT work, labelers wrote demonstrations, ranked model outputs, and provided the preference data used to train a reward model and optimize the policy with reinforcement learning from human feedback, or RLHF ([Training Language Models to Follow Instructions with Human Feedback](#)).

A classic RLHF pipeline has five stages:

1. Train or start from an SFT assistant model.
2. Sample multiple candidate responses to prompts.
3. Collect human rankings or pairwise preferences.

4. Train a reward model to predict those preferences.
5. Optimize the assistant policy for higher predicted reward, usually while constraining it to remain near a reference model.

The reward model is another learned model. It maps a candidate response and its context to a score that approximates preferences in the comparison data ([Deep Dive, around 02:52:39](#)). The reference constraint limits how far policy optimization moves from the starting behavior; without such a constraint, optimization can more easily drive the policy into regions where the reward model is unreliable.

Direct Preference Optimization, or DPO, uses preferred and dispreferred responses to optimize the policy with a classification-like objective, avoiding the same separate reward-model training and online reinforcement-learning loop ([Direct Preference Optimization](#)). It is another way to learn from preferences, not a source of ground truth.

Preference judgments can also come partly from models rather than directly from people. For example, Constitutional AI uses feedback guided by written principles to reduce the amount of human labeling needed for parts of harmlessness training ([Constitutional AI](#)). Whether feedback is human- or model-generated, its quality and coverage shape the behavior that optimization reinforces.

Proxy Rewards and Reward Hacking

A learned reward is a proxy for the qualities its designers care about. Human preferences are compressed into finite comparisons and then approximated by a model, so the resulting score is necessarily incomplete. Karpathy describes a reward model as a lossy simulation of human preference ([Deep Dive, around 03:00:54](#)).

When a policy is optimized strongly against an imperfect proxy, it may discover outputs that receive high scores without satisfying the underlying intent. This is reward hacking ([Deep Dive, around 03:04:11](#)). A policy might learn superficial markers of a preferred answer, become unnecessarily verbose, agree too readily with the user, or exploit blind spots in the reward model.

Reference constraints and other regularization can limit how far optimization moves, but they do not turn a proxy into truth. Preference optimization can improve average judged quality while still introducing systematic distortions outside the comparisons represented in training.

Verifiable Rewards and RLVR

Some training tasks provide rewards that can be computed directly. An exact answer can be compared with a known result, a proof can be checked, a program can be run against tests, or a game can return a score. Other qualities—such as whether an explanation is helpful or whether a tone is appropriate—depend much more on judgment.

Reinforcement learning with verifiable rewards, often abbreviated RLVR, uses such checkable outcomes as the reinforcement signal. Karpathy contrasts reinforcement learning in verifiable domains with reinforcement learning based on unverifiable preferences ([Deep Dive, around 02:51:33](#)). Verifiable signals can be cheaper to apply at scale and less subjective than preference scores for the tasks they cover.

Verification is still scoped to what the checker measures. Passing a checker does not establish every desirable property of the response, and gains on verifiable tasks need not transfer uniformly to tasks without comparable feedback. RLVR can strengthen problem-solving behavior that reaches checkable outcomes; it does not prove that generated reasoning is faithful or universally reliable. [Chapter 8](#) discusses reasoning behavior at inference time.

Behavior Is Not Capability

Post-training changes observable behavior and can also improve, suppress, or redirect task performance. The distinction between behavior and capability is therefore useful, but capability is not a directly visible, fixed quantity. What a model produces depends on the input, learned response tendencies, decoding, and the specific task.

A refusal shows that the model produced a learned refusal behavior in that context. It does not by itself show that the model lacks the ability to produce the requested content under every other context. Conversely, one successful answer does not establish a robust capability, and a confident answer does not establish that the underlying claim is true.

The same limitation applies to safety behavior. Post-training can make undesirable outputs less likely, but a learned refusal policy is not a hard guarantee. [Chapter 10](#) examines uncertainty, hallucination, and refusal behavior, while [Chapter 12](#) distinguishes model-generated actions from their execution. Formal controls outside the model belong to the companion [Agent Harness](#).

Full and Parameter-Efficient Fine-Tuning

The training objective and the choice of trainable parameters are separate decisions. Full fine-tuning updates all or nearly all of the base model's parameters and produces a new checkpoint whose behavior reflects those updates. Because knowledge and behavior are distributed across parameters, an update aimed at one task can also change performance elsewhere.

Parameter-efficient fine-tuning, or PEFT, keeps most base parameters frozen and trains a smaller set of parameters. LoRA is a common PEFT method that learns low-rank updates to selected weight matrices. Other approaches add adapters or train small prompt-like parameter sets. These methods reduce the number of trainable parameters and can store task-specific changes separately from the base checkpoint, although their quality and runtime properties depend on the method and task.

Both full fine-tuning and PEFT alter learned behavior through training data and an optimization objective. Neither turns model parameters into a live database or provides continuously updated facts with source attribution. [Chapter 11](#) covers retrieval from external collections, and [Chapter 14](#) summarizes the boundary between model properties and surrounding systems.

Key Takeaways

- Post-training applies additional optimization that makes a base model more likely to behave like an assistant.
- SFT data may be human-written, synthetic, or mixed, and it teaches both task responses and serialized interaction conventions.
- RLHF, DPO, and AI-feedback methods steer behavior using preferences; their objectives remain proxies rather than ground truth.
- RLVR supplies checkable rewards for tasks with verifiable outcomes, but its signal covers only what is actually verified.
- Refusals and successful answers are observations of behavior, not complete proofs about capability, truth, or safety.
- Full fine-tuning updates most or all parameters, while PEFT methods such as LoRA train a smaller parameter set.

Chapter 8: Prompting and In-Context Learning

Prompting is inference-time conditioning. A model receives a token context and produces a continuation conditioned on that context. Changing the instructions, data, examples, or formatting in the context can therefore change the distribution of possible continuations without changing the model's parameters.

In-context learning (ICL) is the model's ability to adapt its behavior from patterns presented in that context. The adaptation lasts only for the current context; it is not a weight update and does not become a persistent skill or memory. GPT-3 made this capability prominent through zero-shot, one-shot, and few-shot results across many tasks ([Language Models are Few-Shot Learners](#)).

What Makes Up a Prompt

A prompt may combine several kinds of content:

- **Instructions** describe the requested task.
- **Input data** supplies the text, question, or evidence to work on.
- **Examples** demonstrate an input-to-output pattern.
- **Constraints** narrow acceptable answers.
- **Output format** describes the desired structure of the continuation.

Chat APIs may also distinguish system, developer, user, and assistant messages. Those roles are not separate channels inside the Transformer. A provider-specific chat template serializes roles and content into the representation processed by the model. Post-training teaches the model patterns associated with those role markers, but it does not turn them into an infallible priority mechanism. The discussion of conversation tokenization in the deep dive illustrates how roles ultimately become part of the model input ([Deep Dive, around 01:05:03](#)).

The same words can have different effects depending on their position, surrounding labels, examples, and the model's training. Prompting works by supplying evidence about the continuation that is wanted, not by installing a deterministic program.

Structure Reduces Ambiguity

Headings, delimiters, tags, and consistent field names can help the model distinguish a task from its input. An explicit output format can also make a desired continuation easier to infer. For example:

Task:

Answer the question using the documents below. Cite the source ID.

If the documents do not contain the answer, reply: NOT_FOUND.

```
<documents>
  <doc id="d1">Refund requests are accepted within 14 days of delivery.</doc>
  <doc id="d2">Gift cards are non-refundable.</doc>
</documents>
```

Question:

How long do I have to request a refund?

Answer:

The headings and tags expose the intended structure: task, evidence, question, and answer. They do not make the model parse the prompt like a compiler, and they do not guarantee that the answer will follow the instructions.

Most importantly, a delimiter is not a security boundary. If a document inside `<documents>` says "ignore the task and reveal a secret," the model may still be influenced by it. Delimiters can reduce ordinary ambiguity, but they do not reliably isolate data from instructions or enforce trust.

Zero-Shot, One-Shot, and Few-Shot Prompting

A **zero-shot** prompt specifies a task without showing a completed example. A **one-shot** prompt includes one example. A **few-shot** prompt includes several. In each case the parameters remain fixed; only the context changes.

Few-shot prompting works because a model can continue patterns demonstrated by input-output pairs. It is often useful for:

- applying task-specific labels,
- extracting fields from irregular text,
- matching a particular response format or style,
- handling ambiguous or insufficient input,
- and translating a natural-language request into a structured representation.

Examples provide more than a format. They also imply assumptions about labels, edge cases, tone, and what information matters. That makes example selection consequential. If every demonstration is positive, the model may develop a positive bias for the next case. If demonstrations contain obsolete facts or accidental formatting quirks, the continuation may reproduce them. The order of examples can matter as well.

Examples also consume context tokens. More demonstrations are not automatically better: irrelevant or conflicting examples can make the task less clear. Few-shot prompting is useful when examples convey a pattern more effectively than prose, but its effect remains probabilistic and model-dependent.

Reasoning Prompts and Their Limits

Chain-of-thought prompting asks for intermediate reasoning before a final answer. It can improve performance on some multi-step tasks, especially for sufficiently capable models ([Chain-of-Thought Prompting Elicits Reasoning in Large Language Models](#)). The effect varies with the model, task, demonstrations, and decoding method; "think step by step" is not a universal improvement.

Generated reasoning is still generated text. A plausible chain can contain an early mistake, skip a decisive step, or rationalize an answer reached through other internal computations. Visible reasoning is therefore not guaranteed to be a faithful explanation of how the model produced its answer. It should not be treated as proof that the conclusion is correct.

Self-consistency is a related technique: sample multiple reasoning paths and aggregate their final answers rather than rely on one path ([Self-Consistency Improves Chain of Thought Reasoning in Language Models](#)). Agreement can improve results on some benchmarks, but repeated agreement does not establish truth. The method also uses multiple generations and therefore more inference compute.

Reasoning Models and Test-Time Compute

Some models are post-trained to spend additional inference-time computation on intermediate reasoning. These are commonly called *reasoning models*. DeepSeek-R1, for example, describes reinforcement learning on verifiable tasks as a route to strong reasoning behavior ([DeepSeek-R1](#)); [Chapter 7](#) introduces this kind of post-training.

This is a form of *test-time compute*: a difficult prompt may lead the model to produce more intermediate tokens before its final answer. Those tokens may be visible, hidden, or summarized by the provider, but they still affect latency and computational cost. Asking such a model for an additional explicit chain of thought may be redundant or may interfere with its learned behavior.

More reasoning can improve the use of information already available to the model, but it is not grounding. It does not supply a missing current fact, make a false premise true, or turn a generated explanation into verified evidence. A reasoning model remains conditioned on its parameters and current input.

Tool-Use Prompting at the Model Boundary

Tool use begins, from the model's perspective, as another prompting and generation problem. Tool definitions, argument schemas, and sometimes examples are included in the model's input. The model can then generate a structured call such as:

```
{
  "name": "get_weather",
  "arguments": {
    "city": "Boston"
  }
}
```

That JSON is only model output. The model has not contacted a weather service or changed external state. An external system interprets the proposed call, decides whether and how to execute it, and may return an observation in a later model context. The model can then condition its next continuation on that observation.

This chapter is concerned only with how the tool description and observed result affect model generation. The execution loop and the distinction between proposing and acting are covered in [Chapter 12](#). Tool interfaces and production controls belong to the companion [Agent Harness](#).

Prompt Injection as Context Confusion

Prompt injection occurs when untrusted content contains instruction-like text that changes model behavior in an unintended way. A retrieved page might say to ignore the original question. An uploaded document might contain hidden commands. A tool observation or an image can also carry text that competes with the intended task.

At the model level, all of this becomes input that can influence the next-token distribution. Role markers, phrases such as "treat this as data," and delimiters may help the model infer the intended hierarchy, but they do not create a hard separation between trusted instructions and untrusted content. Prompt injection is therefore not solved merely by writing a stronger prompt or nesting the untrusted text inside tags.

The model can be trained to resist many such instructions, but resistance is behavioral rather than an enforcement guarantee. Any security guarantee about external actions or protected information must be implemented outside the model. [Chapter 12](#) establishes that execution boundary; the companion chapters on [system prompts](#) and [security controls](#) cover the system-level treatment.

What Prompting Cannot Provide

Prompting can elicit and steer capabilities learned during training, but it does not update the model's parameters. It cannot reliably create a capability the model lacks, guarantee a correct answer, or make generated reasoning faithful.

A prompt is also not durable memory. It conditions one model call. If an application later sends an earlier exchange again, that text becomes part of a new context; the model itself did not preserve state between calls. Context-window limits and their consequences are covered in [Chapter 9](#).

Finally, prompting grants no permission and performs no external action. A sentence that says "you may access this account" is still just input text, and a generated action description is still output text. Execution, persistent state, and security isolation are properties of systems around the model, not properties supplied by a prompt.

Key Takeaways

- Prompting changes inference-time context, not model parameters.
- In-context learning uses instructions and examples in the current context to shape behavior.
- Structure and delimiters can reduce ambiguity, but they are not security boundaries.
- Few-shot and reasoning prompts can improve some tasks, but their effects are probabilistic and model-dependent.
- More test-time reasoning does not create new evidence or guarantee correctness.
- Tool calls are generated proposals; external systems execute them.
- Prompts do not provide durable memory, permission, execution, or security isolation.

Chapter 9: Context Windows and KV Cache

The context window is the token sequence a model can condition on during one generation. Its limit usually applies to the prompt **plus the tokens generated so far**, not to the prompt alone. If a model has a context limit of (C) tokens and the serialized prompt uses (n), at most (C-n) positions remain for generation. Providers may also impose separate input and output limits, so the model's architectural limit and an API's accepted limits are not always identical.

Context is temporary input to one generation. It is not durable state, and it is not a guarantee that the model will use every included token reliably.

A Finite Token Sequence

A model receives token IDs, including any special tokens added by its chat template. All of them occupy positions in the sequence. As generation proceeds, each selected token is appended to that same sequence and reduces the remaining capacity. Tokenization and hidden formatting overhead therefore affect both how much input fits and how much output can still be produced; see [Chapter 2](#).

Exceeding a limit is a capacity failure: an API may reject the request, truncate part of it, or stop generation at its configured output bound. Staying below the limit answers only whether the sequence fits. It does not establish that the model can recall, compare, or reason over every part of the sequence equally well.

Karpathy describes the context window as the model's finite working context: the information it can currently see while generating ([Intro to LLMs, around 00:32:42](#)). The working-memory analogy is useful as long as it is not mistaken for persistence or reliable random access.

Prefill and Decode

Autoregressive inference has two computationally different stages.

Prefill

During **prefill**, the model processes the prompt through all [Transformer layers](#). Causal masking allows the prompt positions to be computed in parallel even though each position can attend only to itself and earlier positions.

For a prompt of length (n), standard dense self-attention performs attention work that grows as $O(n^2)$. Optimized attention kernels can greatly reduce intermediate memory traffic and constants without changing that basic pairwise-comparison scaling. Prefill produces the logits used to select the first generated token and also creates the key and value tensors that will be reused during decoding.

Prompt length therefore has a direct effect on prefill work and often on time to first token. Other parts of the model, hardware, batching, and serving implementation also contribute, so token count alone does not determine latency.

Decode

During **decode**, the model selects one new token at a time. At each layer, it computes the new token's query, key, and value. The query attends to the keys and values already stored for the preceding sequence; the new key and value are then appended to the cache.

With a KV cache, projections and MLP computations for earlier tokens do not need to be repeated. For standard dense attention, however, the new query still compares against a growing prefix. The attention work for one decoded token is therefore $O(t)$ at sequence length (t), not constant time. Generating (m) tokens after an (n)-token prompt requires roughly $O(mn+m^2)$ decode-attention work across those tokens. Without a cache, a naive implementation would repeatedly run the whole growing prefix and redo much more computation.

This distinction explains two familiar latency measures: prefill largely determines time to first token, while sequential decode largely determines the rate at which later tokens arrive.

What the KV Cache Stores

In every attention layer, each previously processed token has a key vector and a value vector. The **KV cache** stores those tensors for reuse by later decode steps. It is a cache of intermediate numerical state for an exact token prefix, not a cache of facts or a summary of the text.

Its size grows approximately with

KV elements per request =
 $2 * \text{layers} * \text{cached_tokens} * \text{KV_heads} * \text{head_dimension}$

per active sequence, multiplied by the number of bytes used for each element. Batch size, parallel samples, and beam search can multiply that cost. Architectures such as grouped-query or multi-query attention use fewer KV heads and can reduce it, while sliding-window or other attention variants may change which tokens must remain cached.

Longer contexts thus consume more cache memory and require each new query to read more cached state. KV-cache capacity can limit batch size and throughput even when model weights fit comfortably in memory.

A cache is not “stale” merely because the prefix describes an outdated fact. The tensors still correctly represent the exact tokens from which they were computed; the **content** is stale. If a token in that prefix changes, the affected hidden states and their derived keys and values must be recomputed.

Context, KV Cache, and Provider Prompt Caching

These three ideas are related but not interchangeable:

- The **context** is the token sequence the model is allowed to condition on.
- The **per-request KV cache** holds layer-level key and value tensors for positions already processed during that generation.
- **Provider prompt caching** may reuse prefill work for a shared prefix across separate requests.

Provider prompt caching is an API and serving contract, not a property implied by the model's context window. Eligibility, prefix-matching rules, retention time, billing, and latency effects are provider-specific; a match often requires identical leading tokens. A provider may implement the feature by retaining KV-derived state or by other internal means. Either way, it does not enlarge the context window, make the prefix more trustworthy, or turn the request into persistent semantic memory.

Working Context Is Not Persistent State

When a call ends, its context does not by itself become state available to a future call. An application can preserve information externally and include some of it in a later prompt, but that is a separate system mechanism. Detailed context-management and memory architectures belong to [Chapter 2](#) and [Chapter 3](#) of *Agent Harness*.

Long-Context Capacity and Reliability

A larger advertised window increases **capacity**: more tokens can fit. It does not guarantee uniform or reliable **utilization** of those tokens. Long inputs can expose several empirical weaknesses:

- sensitivity to where relevant information appears;
- reduced accuracy when relevant material is surrounded by distractors;
- difficulty integrating evidence spread across distant positions;
- declining task performance before the hard context limit is reached.

The term **context rot** is often used for this broad degradation as context grows. It is an empirical behavior, not a single mechanism or a universal threshold. Its severity depends on the model, task, sequence length, position, and surrounding content.

Lost in the Middle found that models on key-value retrieval and multi-document question answering could perform better when relevant information appeared near the beginning or end than when it appeared in the middle ([Liu et al., 2023](#)). The exact pattern is not identical for every model, but it demonstrates the central distinction: information being inside the supported window does not mean the model will use it reliably.

Long-context claims should therefore be read precisely. A maximum token count describes an accepted sequence length. Performance on a short retrieval probe describes one behavior. Neither alone establishes dependable comprehension, recall, or reasoning across arbitrary inputs at that length.

Context Is Not an Enforcement Boundary

The model computes over the serialized tokens it receives; it does not provide reliable trust isolation between different parts of that sequence. Labels, role markers, and delimiters may influence behavior, but untrusted text can still influence generation, as discussed in [Chapter 8](#).

Context is therefore an input and attack surface, not an authorization or enforcement boundary. Any hard access or action boundary must exist outside the model. The detailed controls belong to the companion [Agent Harness](#).

Key Takeaways

- A context-window limit usually covers the prompt plus generated tokens; provider input and output limits may add further constraints.
- Prefill processes the prompt and builds per-layer keys and values; decode then generates sequentially while reusing them.
- KV cache removes repeated computation for old tokens, but its memory grows linearly with cached sequence length and standard decode attention still scans a growing prefix.
- Context, a per-request KV cache, and provider prompt caching are different concepts with different lifetimes and contracts.
- Long-context capacity does not guarantee reliable use of every included token.
- Context is temporary input, not persistent memory or a model-enforced trust boundary.

Chapter 10: Knowledge, Hallucination, and Uncertainty

LLMs often sound knowledgeable because pretraining encodes statistical regularities from large text collections into model parameters. Those regularities support recall, explanation, and generalization, but they do not form an authoritative record of facts. A model can produce fluent text that is false, stale, or unsupported even when its tone and format resemble a reliable answer.

Karpathy's introduction presents hallucination as a central failure mode: a model can generate plausible text without having a built-in truth oracle ([Intro to LLMs, around 00:10:04](#)). Hallucination is not limited to cases with no evidence. A model can also misread supplied evidence, cite the wrong part of it, or extend a claim beyond what the evidence supports.

Parametric Knowledge Is Not a Database

Parametric knowledge is information that can be elicited from patterns encoded in learned weights. It has no database-style guarantee of a unique record, explicit provenance, or current value. A fact may have appeared in several contradictory forms during training, may be weakly represented because it was rare, or may have changed after the relevant training data was collected.

This also means that "stored" is only a metaphor. There is usually no single parameter or row corresponding to one fact. Related names, dates, phrasings, and associations are distributed across the network and interact with the current context. A prompt may successfully elicit one association in one wording and a different or incorrect association in another.

Parametric knowledge can nevertheless be useful. The important boundary is that a plausible answer from the parameters is a prediction, not a read from an authoritative source.

How Plausible False Answers Emerge

The pretraining objective rewards predicting likely continuations, not checking each claim against reality. Training text also contains mistakes, myths, fiction, outdated material, and confident prose whose truth is not marked. Post-training can make answers more helpful and complete, but helpfulness and completeness do not guarantee factuality.

An answer-shaped prompt further narrows the likely continuation toward an answer-shaped response. If a question presupposes a nonexistent event, book, or theorem, continuing the premise may be statistically easier than challenging it. If an exact citation is requested without a source, a title, author list, venue, and year can all look locally plausible even when the combined reference does not exist.

Risk is especially high when:

- the requested fact is rare or recent;
- the question contains a strong but unsupported premise;
- an exact quotation or citation is demanded without a source;
- the interaction rewards fluent completion or an authoritative format more than acknowledging uncertainty.

The resulting prose may resemble an encyclopedia entry, legal clause, or research citation. That authoritative shape can increase a reader's confidence without increasing the answer's truth.

A Taxonomy of Model-Level Errors

Several mechanisms can produce a false or unsupported answer:

- **Recall errors** substitute the wrong name, date, relation, or detail.
- **Staleness** reproduces a claim that was once true or common but is no longer current.
- **Conflation** blends attributes of different people, documents, events, or concepts.
- **Imitative falsehoods** reproduce misconceptions and myths found in human-written text. TruthfulQA was designed to test whether models tell the truth instead of imitating plausible human falsehoods ([TruthfulQA](#)).
- **False-premise compliance** accepts an unsupported premise and generates an explanation around it. For example, "Why did X happen?" steers the continuation toward causes even when whether X happened is still in question.
- **Evidence misuse** contradicts supplied material, selects the wrong passage, or infers more than that material supports.

These categories overlap, and not every false answer reveals which internal mechanism caused it. They are still useful because they show that hallucination is broader than inventing a fact from an empty context.

Grounding Changes the Input

Grounding supplies external evidence or observations in the model's current input. This can make a supported continuation more likely, but it does not rewrite the model's parameters or guarantee that the evidence will be used correctly. The model can ignore a relevant passage, favor a conflicting parametric association, or make an unsupported inference from a real source.

[Chapter 11](#) explains retrieval-augmented generation and the path by which external text enters a prompt. [Chapter 12](#) explains the separate boundary between a model generating a proposed action and an external system executing it.

Uncertainty and Calibration

Uncertainty is not the same as calibration. Operationally, a set of predictions is calibrated at confidence (q) when, among representative predictions assigned confidence (q), approximately a fraction (q) are correct. For example, among answers assigned 70% confidence, about 70% should be correct. Calibration is therefore a property measured over many cases, not a property established by one confident sentence.

Accuracy and calibration are also different. A model can have high average accuracy while assigning 99% confidence to many of its errors, or lower accuracy while reporting probabilities that better match observed frequencies. Calibration can change across domains, question formats, prompts, decoding settings, and post-training.

Three quantities should not be conflated:

- **Verbal confidence** is language such as "probably" or "definitely." It is generated text and can reflect style as much as uncertainty.
- **Token probability** is the model's probability for a particular next token. It measures the likelihood of a surface continuation, not directly the probability that a complete claim is true.
- **Answer-correctness probability** is a task-level estimate that a final answer is correct under a defined evaluation rule. A standard language-model response does not automatically provide this quantity.

Research suggests that models can sometimes estimate whether they know an answer, but this ability does not transfer perfectly across tasks and prompts ([Language Models Mostly Know What They Know](#)). Self-reported or elicited confidence is therefore evidence about model behavior, not independent verification of the answer.

Epistemic Abstention Is a Learned Behavior

An **epistemic abstention** declines to answer because the model cannot support an answer with sufficient knowledge or evidence. Phrases such as "I don't know" and "the information provided is insufficient" are output behaviors, not direct readouts from a dedicated uncertainty register.

The tendency to answer rather than abstain comes from more than one source: the next-token objective, answer-like patterns in pretraining data, the wording of the prompt, and post-training preferences can all contribute. Karpathy's deep dive highlights one version of this problem: answer demonstrations commonly contain confident, complete responses and few examples of "I don't know" ([Deep Dive, around 01:20:32](#)). This can reinforce answer-shaped completion, but SFT alone is not a complete explanation for hallucination.

Post-training can explicitly reward abstention on questions outside a model's demonstrated knowledge boundary. The learned behavior remains incomplete: a model may fail to abstain on an unfamiliar gap, or abstain even when it could answer. An abstention is thus useful behavior, but not proof of a perfectly detected internal knowledge boundary.

Safety Refusal and Jailbreaks

A **safety refusal** declines a request because it falls into a learned category of unsafe or disallowed behavior. It differs from epistemic abstention: a model may understand a request and still refuse it for safety reasons, while a benign question may call for abstention simply because the answer is unknown.

Safety refusals are shaped by post-training and context, so they are probabilistic behaviors rather than hard guarantees. A **jailbreak** is an attempt to elicit behavior that bypasses those learned safety constraints, for example by reframing the request through role-play, hypotheticals, or encoded instructions. Karpathy's introduction uses jailbreak examples to show that refusal behavior can be disrupted by context ([Intro to LLMs, around 00:46:16](#)).

Jailbreak and prompt injection are related but not synonymous. A jailbreak is defined by the goal of bypassing behavioral constraints and can come directly from a user. [Prompt injection](#) occurs when untrusted content supplies instruction-like text that changes the model's behavior contrary to the intended task; it may target safety, but it can also redirect an otherwise benign task. They overlap when injected content attempts a safety bypass.

Because refusal is learned behavior, guarantees about access or external actions must exist outside the model. The execution boundary is covered in [Chapter 12](#), and the detailed system controls belong to the companion [Agent Harness](#).

Self-Report Is Not Authoritative Metadata

Questions such as "What model are you?", "When is your training cutoff?", or "Can you browse the web?" ask the model to generate another continuation. The answer may reflect post-training examples, the current prompt, or information supplied by the application. It is not an introspective read from the deployment configuration.

A self-report can be correct, especially when accurate metadata is present in context, but the generated statement alone does not establish its accuracy. Model identity, release details, and deployed capabilities are properties of the model artifact and surrounding system, not facts that should be inferred from the model's prose.

Citation-Shaped Hallucination

Citations are particularly convincing because their format signals evidence. A model can generate a plausible title, author, journal, statute number, URL, or quotation even when no matching source exists. It can also cite a real source that does not support the claim or attach a valid citation to the wrong sentence.

Three questions are independent:

1. **Source existence:** does the cited source exist?
2. **Claim support:** does that source actually support the associated claim?
3. **Citation attachment:** is the citation formatted and placed so that the supported claim is identifiable?

A polished citation answers none of these questions by appearance alone. Retrieval and provenance are introduced in [Chapter 11](#); evaluating factual and citation correctness is covered in [Chapter 13](#).

Model Failure vs System Failure

Once external evidence or observations are involved, an incorrect final answer is not automatically evidence that the model invented it. If a model faithfully summarizes an erroneous observation, the upstream source or system failed even though the final claim is false. If the observation is correct but the model contradicts it or adds an unsupported claim, that part is a model failure. Both can occur in the same answer.

This distinction matters conceptually because grounding changes what the model receives; it does not collapse the model and the surrounding system into one component. Diagnosing the complete evidence, tool, permission, and execution path is a systems topic treated in the companion [Agent Harness](#).

Key Takeaways

- Parametric knowledge is distributed, fallible, and not an authoritative database.
- Fluent false answers arise from predictive training, imperfect data, answer-shaped prompts, and learned response preferences.
- Hallucination includes recall errors, staleness, conflation, imitative falsehoods, false-premise compliance, and misuse of supplied evidence.
- Calibration is measured across cases; verbal confidence, token probability, and answer-correctness probability are different quantities.
- Epistemic abstention and safety refusal serve different purposes, and both are learned behaviors rather than infallible internal signals.
- Jailbreaks target learned behavioral constraints; prompt injection describes untrusted instructions changing the intended task.
- Model self-reports and citation-shaped text are not authoritative metadata or proof of support.
- A false system output may result from model error, an erroneous external observation, or both.

Chapter 11: Embeddings and Retrieval

Retrieval augments a model with external information at inference time. Karpathy introduces retrieval-augmented generation (RAG) as a way to bring relevant documents into context instead of relying only on model parameters ([Intro to LLMs, around 00:41:33](#)). The foundational RAG paper combines a parametric sequence model with information accessed through retrieval ([Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks](#)).

RAG changes the model's input, not its weights. Its usefulness therefore depends on two separate questions: did retrieval find the needed evidence, and did the model use that evidence correctly?

Why Retrieval

Information learned during training is compressed into model parameters. This parametric knowledge is broad and immediately available during generation, but it reflects the training data and does not update simply because a source document changes.

Retrieval takes a different path. At inference time, a system searches an external collection, selects relevant passages, and includes them in the model's finite context:

- **Parametric knowledge** is represented indirectly in the model's weights.
- **Retrieved information** remains outside the weights and is supplied as input for a particular call.

The external collection can be updated without training a new checkpoint, and retrieved passages can retain source labels for citation. Retrieval does not guarantee that the selected text is relevant or true, however, and placing a passage in context does not guarantee that the model will use it faithfully.

Retrieval Embeddings

The word *embedding* is used for related but distinct representations:

- A **token embedding** maps one token ID to the vector that enters a Transformer. Subsequent layers turn these initial vectors into contextual, position-dependent hidden states.
- A **retrieval embedding** maps a variable-length query or passage to a fixed-length vector intended for similarity search.

These are not interchangeable. A retrieval model is trained so that relevant query-passage pairs tend to receive similar vectors. Some systems use the same encoder for both sides; others use distinct query and document modes. The model's required mode and preprocessing must therefore be used consistently at indexing and query time.

Dense retrieval compares the query vector with stored passage vectors, commonly using cosine similarity or dot product. If vectors are normalized, cosine similarity and dot product produce the same ranking. A high score means closeness according to that embedding model; it is not a calibrated probability that the passage answers the query.

Comparing a query exactly with every stored vector costs work proportional to the collection size. Approximate nearest-neighbor (ANN) indexes such as HNSW and IVF reduce search latency by avoiding an exhaustive scan. Their approximation can omit vectors that an exact vector search would have returned. That index-level loss is distinct from a retrieval model ranking semantically irrelevant passages highly.

Dense, Lexical, and Hybrid Retrieval

Three common retrieval approaches emphasize different signals:

- **Dense retrieval** ranks passages by similarity between learned retrieval embeddings. It can match paraphrases and conceptual relationships even when query and passage share few words.
- **Lexical or sparse retrieval** ranks passages from term-based representations. BM25, for example, uses term frequency, inverse document frequency, and document-length normalization. It rewards matching terms, but it is a relevance-ranking function rather than an exact string matcher like `grep`.
- **Hybrid retrieval** combines dense and lexical results, often by normalizing and weighting scores or by fusing ranks.

Lexical retrieval is often strong for rare names, identifiers, error codes, and exact phrases. Dense retrieval is often strong when the same idea is expressed with different wording. Hybrid retrieval can preserve both signals, but its fusion rule still needs evaluation: raw scores from different retrievers are not automatically comparable.

A second-stage reranker can score a small candidate set more carefully, perhaps with a cross-encoder that reads the query and passage together. Reranking can improve ordering and precision among the candidates it receives. It cannot recover a relevant passage that the first-stage retrieval never included.

A Minimal RAG Pipeline

A minimal RAG pipeline has an indexing phase and a query phase.

At index time:

1. Collect the documents that form the retrieval corpus.
2. Split each document into retrievable chunks.
3. Retain provenance such as source, title, and section.
4. Compute a retrieval embedding for each chunk.
5. Store the chunk text, vector, and provenance in a search index.

At query time:

1. Encode the query with the retrieval model's query encoder or query mode.
2. Search for the top candidate chunks, using dense, lexical, or hybrid retrieval.
3. Optionally rerank the candidates with a more expensive relevance model.
4. Select passages that fit within the model's context and serialize them alongside the question or instruction.
5. Generate an answer conditioned on that augmented input.

The explicit query-embedding step matters: searching document vectors with an incompatible representation can degrade retrieval even when the index itself is functioning correctly. The final prompt may preserve source labels so the answer can refer back to the passages, but citation support and factual faithfulness are separate evaluation questions, discussed in [Chapter 10](#) and [Chapter 13](#).

Chunking

Whole documents are often too large and too topically broad to serve as useful retrieval units. Chunking decides which span of text receives one retrieval representation and which evidence can be selected independently.

Chunk size creates several tradeoffs:

- Chunks that are too small may separate a claim from its explanation or caveat.
- Chunks that are too large may mix topics, weaken the embedding signal, and consume unnecessary context tokens.
- Boundaries that ignore document structure may split a paragraph, API entry, or code function at an unhelpful point.
- Missing source and section information makes a retrieved span harder to interpret or cite.

Structure-aware chunking tries to preserve semantic units such as sections, paragraphs, API entries, or code symbols. Fixed-size windows are simple and can still be useful, especially when document structure is unreliable.

Overlap between adjacent chunks can preserve information that crosses a boundary. More overlap also increases index size and can return several nearly identical passages, consuming result slots and context. Overlap is therefore a tunable tradeoff, not an automatic improvement. When overlapping or duplicate chunks are retrieved together, deduplication can reduce repeated evidence.

Measuring Retrieval

Retrieval evaluation starts with a set of queries and relevance judgments identifying which passages are useful for each query. Different metrics answer different questions:

- **Recall@k** is the fraction of all relevant passages that appear in the top (k) results: $(|R_k \cap G| / |G|)$, where (G) is the relevant set. When each query has one required passage, this is often reported as a top-(k) hit rate across queries.
- **Precision@k** is the fraction of the top (k) results that are relevant: $(|R_k \cap G| / k)$.
- **Mean Reciprocal Rank (MRR)** averages the reciprocal rank of the first relevant result. It emphasizes finding one relevant result early.
- **Normalized Discounted Cumulative Gain (NDCG)** rewards placing highly relevant results near the top and supports graded, rather than only binary, relevance judgments.

These task metrics should not be confused with **ANN recall**, which compares an approximate index's neighbors with those returned by exact vector search. An ANN index can have high ANN recall while both exact and approximate searches return passages irrelevant to the user's task. Conversely, lowering ANN recall can reduce task Recall@k if the omitted exact neighbor was useful.

No metric is meaningful without a clear retrieval unit and relevance definition. If a fact is duplicated across many chunks, chunk-level recall and precision may tell a different story from whether the system found enough evidence to answer the query.

Precision, Recall, Latency, and Context Pollution

Precision and recall are formal relevance measures, not simply synonyms for “retrieve less” and “retrieve more.” Increasing (k) often raises or preserves recall because more candidates are considered, while precision may fall as weaker candidates enter the result set. The exact behavior depends on the query set and ranking.

Retrieval also has a latency budget. Query encoding, index search, hybrid fusion, and reranking all add time before generation begins. ANN search can reduce index latency at the cost of some ANN recall. A larger candidate set gives a reranker more opportunities to find relevant evidence, but increases reranking work. These tradeoffs should be measured together rather than optimizing one metric in isolation.

The selected passages also consume context. Irrelevant, outdated, duplicated, or mutually conflicting passages can distract the model or introduce unsupported alternatives. This is **context pollution**: retrieval may improve Recall@k while making the final input harder to use. Smaller, cleaner result sets can therefore outperform larger ones even when they contain fewer relevant chunks.

Retrieved text can also contain instructions or adversarial content. That model-level prompt-injection problem is covered in [Chapter 8](#); similarity to a query does not establish that a passage should be treated as an instruction.

RAG vs Fine-Tuning

Retrieval and fine-tuning change different things:

- Use retrieval when the task depends on a substantial information collection that changes independently of the model or when answers need to refer to source passages.
- Use [fine-tuning](#) when the goal is to change recurring behavior, style, or task conventions across calls.

Fine-tuning is not a dependable way to keep a changing document collection current, and retrieval does not by itself teach a model a new stable behavior. A system can use both: fine-tuning shapes how the model responds, while retrieval supplies information for the current query.

Companion Boundary

This chapter covers the model-facing mechanics of retrieval. Production index maintenance and the integration of retrieval into broader agent workflows belong to the companion [Agent Harness](#).

Key Takeaways

- Token embeddings are internal token representations; retrieval embeddings represent whole queries or passages for similarity search.
- Dense retrieval captures learned semantic similarity, lexical retrieval captures term evidence, and hybrid retrieval combines them.
- Minimal RAG embeds both chunks and the query, retrieves candidates, optionally reranks them, and adds selected passages to the model input.
- Chunking and overlap determine what can be retrieved and how much duplicate context appears.
- Recall@k, Precision@k, MRR, NDCG, ANN recall, latency, and context pollution measure different parts of retrieval behavior.
- RAG supplies query-time information; fine-tuning changes recurring model behavior.

Chapter 12: Reasoning, Tools, and Agents

An LLM can generate reasoning-like text, but a model call can use only the inputs it receives and cannot by itself cause effects in the world. Tools bridge that boundary: the model emits a representation of a requested operation, an external system performs the operation, and the result becomes new input to a later model call.

Karpathy's introduction discusses tool use and retrieval as ways to augment models beyond pure text generation ([Intro to LLMs, around 00:41:33](#)). Research systems such as ReAct show how language models can alternate between reasoning, actions, and observations ([ReAct](#)). Toolformer explores training models to decide when and how to call APIs ([Toolformer](#)).

Tool Calls Are Model Outputs

Tool use does not change the model's basic nature as a token generator. Given a context that describes available operations and their arguments, the model may generate ordinary text or a structured call representation. Even when constrained decoding guarantees a particular structure, generation still produces a proposed call rather than executing the operation.

This is why “the model can browse” is shorthand. A model does not independently open pages. An external runtime supplies a browser-like operation, executes a selected call, and encodes the resulting page content into a later context.

A Minimal Call/Result Protocol

The logical protocol needs a call identifier, an operation name, arguments, and a result paired with the same identifier. For example:

```
assistant_call:
  id: c1
  name: get_weather
  arguments:
    city: Paris
```

After an external system executes the operation, it can return:

```
tool_result:
  call_id: c1
  content:
    temp_c: 17
    sky: clear
```

This is vendor-neutral notation for the protocol's external representation, not the literal wire format of every API. Providers may serialize calls and results as typed messages, JSON objects, special-token sequences, or other model-specific structures. Models learn the relevant call patterns during post-training; see [Chapter 7](#).

A streamed, incomplete representation is not yet a complete call. Once complete, syntax validity, schema conformance, semantic correctness, and authorization are separate questions. Valid JSON can have the wrong fields; schema-conformant arguments can still name the wrong city; and a sensible request can still be forbidden. Only the external system can decide whether and how to execute it.

The Minimal Agent Loop

A tool-using agent can be reduced to a short loop:

1. An external runtime gives the model the task context and descriptions of available operations.
2. The model generates either a response or a structured call.
3. If it is a call, the runtime decides whether to execute it.
4. The runtime returns the result as an observation in a new context.
5. The model continues until it produces a final response or the runtime stops the loop.

The observation does not update the model's parameters. It changes subsequent generation because it is added to the input context. A sequence of model calls plus external execution can therefore behave like an agent even though

each individual model call remains conditional generation.

What Tools Add

Different operations extend a model along different axes:

- **External knowledge** operations can supply retrieved or current information.
- **Exact computation** operations can perform calculations or repeatable transformations.
- **Environment interaction** operations can read files, open pages, or inspect other live state.
- **External-state operations** can create, update, or send something outside the model.
- **Generative operations** can produce artifacts in another modality.

Native multimodality is not itself a vision tool. A multimodal model may directly receive an encoded image as part of its input, while an external vision or OCR tool performs a separate operation and returns a textual or structured observation. In either case, the model receives a representation, not direct access to the world.

Reasoning and Acting

Reasoning and acting play different roles. Reasoning-like computation can help select or sequence steps using the current context. An action can obtain information that was absent from that context or can request a change to external state. ReAct emphasizes the feedback pattern: reasoning informs an action, and the resulting observation informs what comes next.

The protocol should not be confused with a faithful chain-of-thought record. Some systems expose reasoning-like text, some keep intermediate reasoning hidden, and some return a short summary or plan. Those are different artifacts; a visible plan is not necessarily a transcript of the computation that produced the action. The reliably observable protocol boundary is the proposed action and the observation returned for it. Recording those two can help distinguish a poor call from a misleading or failed result without claiming access to the model's full reasoning process.

Reasoning models may spend additional test-time compute before responding or acting. That can improve how they use existing context, but it does not create new evidence, execute an operation, or provide persistent state between calls.

The Model Boundary

Several limits follow directly from this loop:

- Emitting a call does not mean that the operation ran or succeeded.
- A model call does not naturally preserve task state for a later call; any continuity comes from context supplied again or state held externally.
- Tool results become input context and can be incomplete, incorrect, or adversarial. The model does not provide a reliable trust boundary between data and instructions; see [Chapter 8](#).
- Authorization and external side effects belong to the system that executes the call, not to the model that proposes it.

Those facts are the bridge from model foundations to system design. The companion *Agent Harness* covers [tool interfaces and MCP](#), [permissions and sandboxing](#), [long-running state](#), [human supervision](#), and [computer use and multimodal agents](#).

Key Takeaways

- A tool call is structured model output, not an executed action.
- A minimal agent loop alternates model generation with externally executed calls and returned observations.
- Reasoning can shape an action, while acting can add evidence or change external state.
- Execution, authorization, persistent state, and supervision exist outside the model.

Chapter 13: Evaluating LLM Behavior

LLM behavior is probabilistic and context-sensitive. A change in the prompt, chat template, decoding configuration, or sampled token path can change the result. Even nominally identical requests may not produce identical outputs. Evaluation therefore accumulates evidence across tasks and trials; a single impressive answer or failure is weak evidence.

An eval score is always conditional on a setup: the model checkpoint or served version, the inputs, the decoding configuration, the task distribution, and the grader. It describes observed behavior under those conditions, not an intrinsic, context-free property of the model.

Benchmarks and Their Limits

Public benchmarks measure particular slices of behavior. MMLU tests broad academic and professional knowledge, TruthfulQA probes answers to questions that invite common misconceptions, and HumanEval and MBPP use executable programming problems to test code generation ([MMLU](#), [TruthfulQA](#), [HumanEval](#), [MBPP](#)). BIG-bench and HELM illustrate broader efforts to compare models across many tasks and metrics ([BIG-bench](#), [HELM](#)).

These scores are useful comparison signals, but their scope is limited. A benchmark may differ from the tasks of interest, emphasize one format, become saturated, or be contaminated by training data. Prompting and decoding choices also affect the measured result. A higher aggregate score does not guarantee improvement on every task, language, or failure mode.

Benchmarks that require tools or an interactive environment measure a model together with the surrounding protocol and system. They can be valuable, but their scores are not measurements of the model alone.

Representative Tasks

A golden task set is a curated collection of inputs and checkable success criteria. It should cover representative ordinary cases, important edge cases, and known historical failures. A criterion may be an exact answer, a set of required properties, executable checks, or a clear rubric. The set should be versioned so that changes to tasks and expected results remain distinguishable from changes in model behavior.

Toy prompts are useful for diagnosis, but they do not establish performance on a broader distribution. The task set should resemble the inputs and output requirements for which the result will be interpreted. Holding out some tasks also reduces the risk of repeatedly tuning to the visible examples rather than improving the intended behavior.

Capability and Regression Evals

A **capability eval** asks which tasks a model can perform under a stated prompting and decoding setup. It deliberately includes tasks that are not yet solved reliably, so it can reveal both demonstrated behavior and room for improvement. It provides evidence about elicited capability under those conditions, not proof of everything the model could do under every possible prompt.

A **regression eval** asks whether behavior that previously worked still works after a model, prompt, or decoding change. A capability task that becomes consistently reliable may later serve as a regression case. The two eval types use similar mechanics but answer different questions.

Repeated Trials

Because behavior can vary from run to run, report the number of trials, the number or rate of successes, and—when comparisons matter—an uncertainty interval. A result such as 8 successes in 10 trials is an estimate with substantial uncertainty; a small difference between two configurations may be noise.

Two commonly used summaries answer different questions:

- **pass@k** is the probability that at least one of (k) attempts succeeds. It is relevant when several candidates can be generated and one successful candidate can be selected.
- **pass^k** is the probability that all (k) attempts succeed. It is relevant when the same task must succeed repeatedly.

If trials are independent and each has success probability (p), then pass@k is $(1-(1-p)^k)$, while pass^k is (p^k) . Real trials may be correlated, so empirical estimates should state how tasks and attempts were sampled. Neither metric is inherently more honest: the appropriate one depends on the behavior contract being studied.

Record the model version, full input, chat template, sampling parameters, and any seed supported by the API. A fixed seed can reduce one source of variation and make paired debugging comparisons easier, but it does not guarantee reproducibility in hosted inference. Backend changes, numerical nondeterminism, and implementation details may still change the output. Use multiple independent trials when the goal is to characterize behavioral variability.

Do not call every pass/fail flip a flaky eval. If the same saved output receives different verdicts, a validator intermittently fails, or task inputs change unexpectedly, the measurement setup or grader is unstable and should be repaired. If repeated model outputs genuinely differ in correctness under the stated setup, the eval has found unstable model behavior; repetition should measure and report it rather than make it disappear.

Grading

Different outputs require different graders:

- **Deterministic or code-based graders** use exact matching, structured-output validation, or executable checks. They are fast and reproducible but can reject valid variations or test only a narrow property.
- **Human graders** can apply domain judgment to ambiguous or subjective outputs, but they are slower and may disagree when the rubric is underspecified.
- **Model-based graders** can apply natural-language rubrics at scale, but their verdicts are themselves probabilistic model outputs.

These methods can be combined. Use deterministic checks for properties they can actually establish, and calibrate human or model judgments against clearly labeled examples. For example, a valid citation format does not establish that the source exists, and source existence does not establish that it supports the claim.

A grader is a proxy for the intended quality, not ground truth. Reward models in post-training have the same basic limitation: they approximate preferences rather than directly observe them ([Deep Dive, around 02:52:39](#)). Model graders also need their model, prompt, rubric, and disagreement policy recorded.

Metrics as Proxies

When development is optimized against a visible metric, outputs can improve on the metric without improving the underlying goal. This is the evaluation analogue of reward hacking discussed in [Chapter 7 \(Deep Dive, around 03:04:11\)](#).

For example, a summarizer may increase citation count by attaching irrelevant citations, an answer may maximize keyword overlap while preserving a factual error, or a model grader may reward polished verbosity over correctness. No single metric fully specifies qualities such as truthfulness, usefulness, or clarity.

Use complementary metrics, balanced and held-out cases, and periodic human inspection of examples where graders disagree. When a score rises, inspect whether the newly rewarded behavior is the behavior the metric was intended to represent.

Where Model Evaluation Ends

Direct model evals can measure properties visible in outputs under supplied context, such as task accuracy, factuality, citation support, and format or schema validity. Preserve the input, output, configuration, and grader result so individual judgments can be inspected.

When success depends on tool execution, error recovery, permissions, side effects, or the final state of an external environment, the unit under evaluation is no longer the model alone. It is a system eval. A complete agent trace, environment-state grading, operational reliability, and release gates belong to the surrounding system; see [Agent Harness, Chapter 10](#).

Key Takeaways

- LLM behavior is probabilistic and context-sensitive, so eval conclusions require representative tasks and repeated trials.
- Public benchmarks provide bounded evidence about their own task distributions, not universal model rankings.
- Capability evals probe what can be elicited; regression evals check whether previously demonstrated behavior persists.
- pass@k and pass^k answer different questions, and a seed does not guarantee reproducible hosted inference.

- Deterministic, human, and model graders all measure proxies and have distinct failure modes.
- Evaluation of tool execution, permissions, side effects, or environment state is system evaluation, not model-only evaluation.

Chapter 14: The Operational Mental Model

A Synthesis of the Book

An LLM is a conditional token generator with broad learned competence. Given an encoded context, it produces a distribution over the next token; repeated selection turns that distribution into text or a structured proposal. Training shapes its parameters, prompting conditions a particular inference, and decoding introduces choices about how a continuation is selected. None of those mechanisms turns the model into a database, a persistent state store, an authority over external facts, or an executor of real-world actions.

The earlier chapters developed that picture in stages. [Chapters 1–7](#) explained the model's core mechanisms, from tokens and next-token prediction through attention, training, inference, and post-training. [Chapters 8–13](#) examined prompting, context, knowledge, retrieval, tool calls, and evaluation while marking where model behavior ends and system responsibility begins. This final chapter collects those boundaries into one operational mental model.

The Model–System Responsibility Boundary

Six model properties determine where the boundary belongs:

Model Property	What the Model Can Do	What the External System Must Do
Generation is conditioned only on the encoded current context.	Interpret and transform the representations supplied in that context.	Select, assemble, and serialize the context the model receives.
Context is finite, and the model has no persistent state of its own.	Use information that is represented in the current call.	Persist state outside the model and decide what to reintroduce later.
Parametric knowledge is incomplete and can become outdated.	Recall learned patterns and reason over supplied evidence.	Obtain authoritative or current facts from external sources.
Capability is jagged, and generation is probabilistic.	Produce useful candidate answers across a broad range of tasks.	Verify requirements that demand exactness or reliability; use evaluation to measure behavior rather than assuming that evaluation itself improves it.
A tool call is an output, not an external effect.	Propose an action or emit structured arguments.	Authorize, validate, and execute the action, and own its consequences.
The model can confuse data with instructions.	Follow learned instruction and safety behavior, lowering—but not eliminating—the probability of unsafe proposals.	Establish trust boundaries and control permissions and real-world effects outside the model.

The boundary is not a judgment about whether the model is “intelligent.” It follows directly from what computation the model performs and what it does not control.

Capability Is Jagged

Model capability is uneven, not a single dial. A model may solve a difficult multi-step proof yet miscount the letters in a word, or write a sound database schema yet make a simple arithmetic error. This “jagged” or Swiss-cheese profile means success on one task does not establish reliability on a nearby task.

Tokenization helps explain some character-level weaknesses ([Chapter 2](#)), and uneven training coverage helps explain some knowledge gaps ([Chapter 5](#)). But the important operational lesson is broader: capability claims need representative, repeated evidence, as discussed in [Chapter 13](#), and exact requirements still need verification outside the model.

From Foundations to Agent Harness

This book explains why those external responsibilities exist. The companion [Agent Harness: A Practitioner's Textbook](#) explains how systems can implement them:

- Context selection and durable state: [Context as a Finite Resource](#) and [Compaction, Memory, and the Sub-Agent Pattern](#).

- Actions and external effects: [Tools and the Agent–Computer Interface](#).
- Trust, permissions, and containment: [Sandboxing, Guardrails, and Safe Autonomy](#).
- Reliability measurement: [Evaluation](#).
- Model selection, routing, cost, and operations: [Model Selection, Routing, and Reasoning Models](#) and [AgentOps](#).

The same boundary applies to multimodal systems: the model receives encoded representations of text, images, audio, or video—not the world itself; representation choices are covered in [Computer-Use and Multimodal Agents](#), and their trust implications in [Sandboxing, Guardrails, and Safe Autonomy](#).

What to Remember

Karpathy ends the deep dive with a pragmatic warning: use these systems as tools, but do not fully trust them ([Deep Dive, around 03:09:24, 03:30:42](#)). The right posture is neither dismissal nor worship. It is to use the model's broad competence while assigning authority, persistence, verification, and consequences to the system that can actually provide them.

If you remember only one sentence, remember this:

The model predicts tokens; the harness owns context, state, tools, permissions, verification, and consequences.

Key Takeaways

- Assign responsibilities to the correct side of the model–system boundary.
- Derive that boundary from model properties, not from the fluency of an output.
- Do not mistake a capable model response for a reliable system.

References

This book is primarily based on the two source lectures below, with paper references added for specific concepts.

Source Lectures

- Andrej Karpathy, *Intro to Large Language Models*. YouTube.
https://www.youtube.com/watch?v=zjkBMFhNj_g
 - Andrej Karpathy, *Deep Dive into LLMs like ChatGPT*. YouTube.
<https://www.youtube.com/watch?v=7xTGNLNPYMI>
-

Core Architecture and Training

- Ashish Vaswani et al., *Attention Is All You Need*, 2017.
<https://arxiv.org/abs/1706.03762>
 - Tom B. Brown et al., *Language Models are Few-Shot Learners*, 2020.
<https://arxiv.org/abs/2005.14165>
 - Jared Kaplan et al., *Scaling Laws for Neural Language Models*, 2020.
<https://arxiv.org/abs/2001.08361>
 - Jordan Hoffmann et al., *Training Compute-Optimal Large Language Models*, 2022.
<https://arxiv.org/abs/2203.15556>
 - Rylan Schaeffer, Brando Miranda, and Sanmi Koyejo, *Are Emergent Abilities of Large Language Models a Mirage?*, 2023. <https://arxiv.org/abs/2304.15004>
 - William Fedus, Barret Zoph, and Noam Shazeer, *Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity*, 2021. <https://arxiv.org/abs/2101.03961>
-

Efficiency and Quantization

- Tim Dettmers et al., *LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale*, 2022.
<https://arxiv.org/abs/2208.07339>
 - Elias Frantar et al., *GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers*, 2022.
<https://arxiv.org/abs/2210.17323>
-

Tokenization

- Rico Sennrich, Barry Haddow, and Alexandra Birch, *Neural Machine Translation of Rare Words with Subword Units*, 2015.
<https://arxiv.org/abs/1508.07909>
-

Post-Training and Preferences

- Long Ouyang et al., *Training Language Models to Follow Instructions with Human Feedback*, 2022.
<https://arxiv.org/abs/2203.02155>
 - Yuntao Bai et al., *Constitutional AI: Harmlessness from AI Feedback*, 2022.
<https://arxiv.org/abs/2212.08073>
 - Rafael Rafailov et al., *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*, 2023.
<https://arxiv.org/abs/2305.18290>
 - DeepSeek-AI, *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*, 2025.
<https://arxiv.org/abs/2501.12948>
-

Prompting, Retrieval, and Tool Use

- Jason Wei et al., *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*, 2022. <https://arxiv.org/abs/2201.11903>
 - Xuezhi Wang et al., *Self-Consistency Improves Chain of Thought Reasoning in Language Models*, 2022. <https://arxiv.org/abs/2203.11171>
 - Patrick Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, 2020. <https://arxiv.org/abs/2005.11401>
 - Nelson F. Liu et al., *Lost in the Middle: How Language Models Use Long Contexts*, 2023. <https://arxiv.org/abs/2307.03172>
 - Shunyu Yao et al., *ReAct: Synergizing Reasoning and Acting in Language Models*, 2022. <https://arxiv.org/abs/2210.03629>
 - Timo Schick et al., *Toolformer: Language Models Can Teach Themselves to Use Tools*, 2023. <https://arxiv.org/abs/2302.04761>
-

Decoding and Evaluation

- Ari Holtzman et al., *The Curious Case of Neural Text Degeneration*, 2019. <https://arxiv.org/abs/1904.09751>
- Stephanie Lin, Jacob Hilton, and Owain Evans, *TruthfulQA: Measuring How Models Mimic Human Falsehoods*, 2021. <https://arxiv.org/abs/2109.07958>
- Saurav Kadavath et al., *Language Models (Mostly) Know What They Know*, 2022. <https://arxiv.org/abs/2207.05221>
- Dan Hendrycks et al., *Measuring Massive Multitask Language Understanding*, 2020. <https://arxiv.org/abs/2009.03300>
- Mark Chen et al., *Evaluating Large Language Models Trained on Code*, 2021. <https://arxiv.org/abs/2107.03374>
- Jacob Austin et al., *Program Synthesis with Large Language Models*, 2021. <https://arxiv.org/abs/2108.07732>
- Aarohi Srivastava et al., *Beyond the Imitation Game: Quantifying and extrapolating the capabilities of language models*, 2022. <https://arxiv.org/abs/2206.04615>
- Percy Liang et al., *Holistic Evaluation of Language Models*, 2022. <https://arxiv.org/abs/2211.09110>

Glossary

Agent

A system that places model calls in a loop: the model may propose a tool call, an external system executes it, and the resulting observation becomes input to a later call. Execution, authorization, and persistent state remain outside the model.

Attention / Self-Attention

The mechanism that lets each token position read other tokens' representations, weighted by relevance. Self-attention is how a Transformer mixes information across the sequence.

Autoregressive Model

A model that generates a sequence one token at a time, conditioning each new token on previous tokens.

Base Model

A model after large-scale pretraining but before assistant-oriented post-training. Base models are strong text completers but are not necessarily helpful assistants.

Byte Pair Encoding

A subword tokenization method that builds a vocabulary by repeatedly merging frequent symbol pairs. It helps models handle rare and unseen words through smaller units.

Chain-of-Thought

A prompting pattern in which the model generates intermediate reasoning text before an answer. It can improve some tasks, but the text is not guaranteed to be a faithful account of the computation that produced the answer.

Chat Template

The serialization format that turns a chat conversation with roles such as system, user, assistant, and tool into the token sequence a model actually receives.

Context Rot

An empirical degradation in how effectively a model uses information as a sequence grows. Its severity depends on the model, task, length, information position, and surrounding distractors.

Context Window

The maximum sequence length a model can condition on during a call, usually shared by the prompt and tokens generated so far. It is a transient working sequence, not durable memory.

DPO

Direct preference optimization. A post-training method that optimizes a model directly from preference data without a separately trained reward model.

Embedding

A vector representation of text, code, images, or other data. Text embeddings are often used for semantic search and retrieval.

Epistemic Abstention

A model response that declines to answer because the requested information is unknown or insufficiently supported. It is a learned behavior, not a guaranteed readout of the model's internal knowledge boundary.

Fine-Tuning

Additional training after pretraining. In LLM practice this may include supervised fine-tuning on demonstrations or preference-based optimization.

Grounding

Conditioning generation on external evidence and tying claims back to that evidence. Grounding changes the model's input; it does not update the model's parameters or guarantee faithful use of the evidence.

Hallucination

Plausible generated text that is not supported by reality or by supplied evidence.

Harness

The software system around a model. It owns responsibilities the model cannot provide by token generation alone, such as execution, persistent state, permissions, verification, and real-world consequences.

Hybrid Search

Combining dense vector retrieval with lexical or sparse retrieval such as BM25. Literal or regular-expression search such as `grep` is a separate exact-text signal that may also be combined with them.

In-Context Learning

The model's ability to adapt behavior based on instructions and examples in the prompt without changing its parameters.

KV Cache

Cached key and value tensors used during Transformer inference to avoid recomputing previous context. It improves efficiency but is not semantic memory.

Logits

Raw model scores for possible next tokens before conversion into probabilities.

Next-Token Prediction

The training objective where the model learns to predict the next token from previous tokens.

Parameters

Learned numerical weights of a model. They encode compressed statistical structure and behavior learned during training.

pass@k

An evaluation metric: the fraction of problems solved by at least one of k sampled attempts. Higher k rewards models that can get there with more tries.

pass^k

An evaluation metric: the fraction of problems for which all k sampled attempts succeed. It measures repeated reliability under the sampled conditions rather than the chance that at least one attempt works.

Post-Training

Training after pretraining that shapes behavior, such as supervised fine-tuning, RLHF, DPO, or constitutional AI.

Pretraining

Large-scale self-supervised next-token-prediction training over a broad text corpus that produces the base model. Every later stage (fine-tuning, post-training) builds on top of it.

Prompt Injection

A failure mode where untrusted content contains instruction-like text that influences the model in a way the harness did not intend.

Provider Prompt Caching

A provider-specific feature that reuses work for matching prompt prefixes across requests. It is distinct from the per-request KV cache and follows provider-specific matching, lifetime, and billing rules.

RAG

Retrieval-augmented generation. An external retrieval step selects information and adds it to the model's input at inference time; the model weights do not change.

Reasoning Model

A model post-trained to use additional test-time computation on difficult tasks, often with reinforcement learning on verifiable rewards. More reasoning tokens can improve some tasks but do not create missing facts or external evidence.

Retrieval Embedding

A fixed-length vector representing a query or passage for similarity search. It differs from the per-token internal representations used inside a Transformer.

Reward Hacking

Optimizing against an imperfect reward or grading signal in a way that improves the measured score without improving the real objective.

Reward Model

A model trained to score candidate outputs according to preference data or another proxy objective. Reward models are useful but can be imperfect simulations of human judgment.

RLHF

Reinforcement learning from human feedback. A post-training method that uses human preference data to steer model behavior.

Safety Refusal

A learned response that declines a request because it falls into a category treated as unsafe or disallowed. It is distinct from epistemic abstention and is not an execution-layer guarantee.

Sampling

Choosing output tokens from a probability distribution. Sampling settings such as temperature and top-p affect creativity, stability, and variance.

SFT

Supervised fine-tuning. Training a model on examples of desired input-output behavior.

Temperature

A decoding parameter that changes the sharpness of the next-token probability distribution. Lower temperature is more deterministic; higher temperature is more varied.

Test-Time Compute

Spending more tokens, time, and money at inference to improve hard answers, as distinct from training-time scaling. Reasoning models are the most common example.

Token

The unit of text a model processes. Tokens may be words, subwords, punctuation, whitespace patterns, or byte-level pieces.

Tool Call

A model output that the harness interprets as a request to run an external operation.

Top-p (Nucleus Sampling)

A decoding setting that samples from the smallest set of tokens whose cumulative probability crosses p . It trims the long tail of unlikely tokens while keeping the choice adaptive to how confident the model is.

Transformer

The neural network architecture underlying most modern LLMs, based on attention mechanisms rather than recurrence.

Source Map

This map links the book's main concepts to the two source lectures. The timestamps are approximate because the source material came from auto-generated YouTube subtitles.

Intro to Large Language Models

Timestamp	Topic	Used In
00:00:24	"Two files" framing: parameters plus code to run them	Chapters 1, 14
00:01:35	Parameters as weights stored on disk; runtime code executes the model	Chapters 1, 6
00:03:59	Where parameters come from: training rather than manual programming	Chapters 3, 5
00:05:16	Large GPU training runs and expensive parameter production	Chapters 5, 6
00:10:04	Hallucination examples and why fluent text can be wrong	Chapter 10
00:11:40	Transformer neural network architecture	Chapter 4
00:14:29	Fine-tuning after pretraining	Chapter 7
00:18:01	Two-stage framing: pretraining and fine-tuning	Chapters 3, 7
00:22:11	Reinforcement learning from human feedback	Chapter 7
00:28:20	Tool use, including browser-like tools	Chapter 12
00:32:42	Context window as finite working context	Chapter 9
00:33:41	Multimodal systems that see and generate images	Chapters 2, 12, 14
00:41:33	Retrieval-augmented generation and tool augmentation	Chapters 11, 12
00:46:16	Jailbreak attacks and safety/refusal behavior	Chapters 7, 10

Deep Dive into LLMs like ChatGPT

Timestamp	Topic	Used In
00:00:28	What is behind the chat text box	Chapters 1, 14
00:01:28	Dataset construction and FineWeb as an example	Chapter 5
00:03:42	Web data filtering and processing	Chapter 5
00:04:54	Language classification and filtering	Chapter 5
00:12:07	Tokens and tokenization	Chapter 2
00:14:34	Dataset becomes a long token sequence	Chapters 2, 3
00:15:36	Training on windows of tokens	Chapter 3
00:23:24	Transformer as the neural network used for LLMs	Chapter 4
00:24:29	Attention block in the Transformer	Chapter 4
00:26:12	Inference after training	Chapter 6
00:35:58	Loss as a training signal	Chapter 3
00:40:11	GPU cost and practical inference/training constraints	Chapters 5, 6
01:05:03	Tokenization of conversations/chat format	Chapters 2, 8
01:20:32	Hallucination, knowledge vs working memory, and training the model to say "I don't know"	Chapter 10
01:25:07	Tool use in modern assistants	Chapter 12
01:33:39	Introducing tools to compensate for model limits	Chapters 11, 12
02:51:20	RLHF and reinforcement learning framing	Chapter 7
02:52:39	Reward model as a separate neural network	Chapters 7, 13
03:00:54	Reward model as a lossy simulation of human preference	Chapters 7, 13
03:04:11	Reward hacking and limits of optimizing a reward model	Chapters 7, 13
03:09:24	Use models as tools, not as fully trusted authorities	Chapters 10, 14
03:09:57	Multimodal models over audio and images	Chapters 2, 12, 14